

The Adaptive Company

A Practical Operating System for Human-Agent Work

Gabriel Sarmiento

Contents

	How to Read This Book	4
	Preface: The Company Is the Design Problem	5
I	Part I: The Constraint Moved	
1	When Production Gets Cheap	7
2	Hidden Operational Debt	12
3	Why Tools Do Not Change the Operating Model	17
II	Part II: The ATOM Model	
4	ATOM: Intent In, Evidence Out	21
5	Context Is Infrastructure	25
6	Design the Decision Before the Agent	29
7	Human-Agent Execution	34
III	Part III: How the System Runs	
8	A Week Inside an Adaptive Company	40
9	Governance in the Work	44
10	Observe, Prove, Learn	47
IV	Part IV: Build the First Loop	
11	Choose a Decision, Not an AI Project	52
12	Build the Minimum Viable Operating Loop	55
13	Prove Value Before You Expand	59
V	Part V: Scale Without Losing the Plot	

14	Fractal Units, Not Fractal Bureaucracy	63
15	Allocate Capital, Talent, and Knowledge Dynamically	67

VI	Part VI: Leadership and Control
----	--

16	Agent Operations: Control at Scale	71
17	The Manager After Coordination Gets Cheap	75
18	Accountability Cannot Be Delegated	79
	Bibliography	85

How to Read This Book

This book is built around one operating problem: AI increases the amount of analysis and execution a company can produce, but production is rarely the only constraint. Decisions, context, interfaces, control, and learning determine whether that new capacity becomes value.

Part I identifies the shift and shows how Hidden Operational Debt can be observed in customer conversations, workflow records, software, and decision traces. Part II defines ATOM - the Adaptive Transformation Operating Model - as a loop from intent through evidence and learning. Part III shows the loop operating with real stops, exceptions, and human judgment. Part IV gives a bounded path from pilot selection through a thirty-day build and a ninety-day evidence gate. Parts V and VI address multi-business scale, shared resources, agent operations, management, and accountability.

The book is explicit when introducing research evidence, a real documented practice, an illustrative example, or an operating hypothesis. Those distinctions matter. A useful model should not borrow certainty from a company story or present a plausible scenario as evidence.

You do not need to implement the whole model. Choose one recurring decision that crosses a painful boundary. Read the book with that decision in mind. By the Field Kit, you should be able to specify its intent, context, authority, execution, evidence, controls, and learning cycle on one page.

Preface: The Company Is the Design Problem

There is a point in a growing company when being the person who can solve anything becomes a liability.

The founder-COO has hired strong people. A Head of Operations now runs delivery. Product, finance, and commercial leaders own their functions. Yet the consequential decisions still return to the founder: a non-standard price, a senior hire, a promise to a strategic customer, a conflict between the established business and a new one.

The company has delegated tasks. It has not transferred the context and authority required to make decisions.

AI makes this condition easier to see and easier to worsen. Teams can produce analysis, software, campaigns, and operating plans much faster. Agents can increasingly act in business systems. But the company still has to decide what matters, which evidence to trust, where authority sits, and who answers when the result is wrong. Faster production sends more work toward any unresolved boundary.

This book is for the operating leader at that boundary: usually a founder, COO, or transformation executive helping a company move from a few dozen people toward a real management system, often while adding products, business units, or geographies. The problem is not simply adoption. Most readers already use AI. The problem is building a company that can convert human and machine intelligence into sound decisions without routing every exception through one person.

ATOM - the Adaptive Transformation Operating Model - is the model developed in these pages. It is a hypothesis about how such a company can operate. It joins intent, context, decisions, human-agent execution, evidence, and learning under explicit controls. It is not a software product, an org chart, or a promise of autonomous management.

The standard throughout the book is practical. Where a claim rests on research, the source is named. Where a company practice is public, it is identified as a real case. Where the narrative combines familiar patterns, it is labeled as an illustrative example. The reader should be able to disagree with the model, alter it, and still use its parts.

The central claim is simple: when production becomes cheap, company design becomes the constraint. The founder's next job is not to make more decisions. It is to design a company capable of making them.

I

Part I: The Constraint Moved

1	When Production Gets Cheap	7
1.1	A real change, stated narrowly	7
1.2	Productivity belongs to a system	8
1.3	The constraint moves	9
1.4	Cheap creation has an option cost	10
1.5	Won't agents absorb the new queues?	10
1.6	What to do on Monday	10
2	Hidden Operational Debt	12
2.1	The company is leaving a trail	12
2.2	Debt has an observable fingerprint	13
2.3	Real practice: agent builders trace the path	13
2.4	The dashboard should show strain, not activity	14
2.5	Illustrative example: a new business line seen through its traces	15
2.6	Ask AI to build the first version	15
2.7	From reporting to an operating instrument	16
3	Why Tools Do Not Change the Operating Model	17
3.1	What an operating model contains	17
3.2	The paved cow path	18
3.3	Local output can create system load	18
3.4	Bottom-up adoption still matters	19
3.5	The test	19

1. When Production Gets Cheap

A common early failure caused by cheap production is not bad output. It is a company that produces more choices than it can absorb.

Consider a founder-COO in a company of forty people. The business is preparing to add a second activity, hire another thirty people, and build its first real layer of management. A Head of Operations is taking over the daily run. This should release the COO to work on structure, capital allocation, and growth.

It does not.

The new business lead can prepare a market analysis in hours. Finance can generate several pricing and margin scenarios. Product and engineering can prototype different offers in parallel. Each team arrives better prepared. Each also arrives with a decision that crosses the boundaries of the company: which assets remain shared, which capability belongs inside the new unit, what risk the center will carry, and what evidence justifies further investment.

The Head of Operations can run the process. Only the COO still holds enough of the company in their head to resolve the tradeoffs. Work has been delegated. Decisions have not.

This is an **illustrative composite**, not a report about a particular company. Its mechanism is common enough to recognize. At forty people, the founder's intervention can be the shortest path to a good answer. During the move to seventy, then perhaps one hundred and fifty, the same habit becomes part of the architecture. Teams learn to escalate because the founder is fast, informed, and exacting. Managers learn that initiative is welcome until it conflicts with an unstated preference. Shared functions accumulate exceptions. New ventures remain attached to the person who launched them because nobody designed the conditions for transfer to the run.

AI can intensify this dependency. It lowers the effort required to produce an analysis, option, plan, prototype, or recommendation. It does not automatically distribute the authority to choose among them. The organization has not shortened the path from intent to result. It has moved the queue toward judgment, integration, and commitment.

1.1 A real change, stated narrowly

Speed has no organizational meaning until we specify where it occurs and what constrains the complete result. A model can answer in seconds while the company still moves through the same decision, integration, and acceptance cycles. The useful question is not "How fast is AI?" It is "Which constraint moves when this work becomes cheaper?"

The defensible claim is narrow. The marginal cost and latency of producing *some* cognitive and software artifacts have fallen sharply. The emphasis belongs on *some*. The effect varies by task, quality threshold, workflow, and consequence.

One useful indicator is the price of model inference at a fixed level of benchmark performance. Stanford's 2025 AI Index reported that the price for a model performing around the GPT-3.5 level on MMLU fell from about \$20 per million tokens in November 2022 to \$0.07 in October 2024—a decline of more than 280 times [1]. Epoch AI, whose data informed that analysis, found very different rates across tasks and performance thresholds: roughly ninefold to nine-hundredfold annual declines in the cases it examined [2]. The spread matters. There is no single "price of intelligence," and the fastest historical decline should not be projected indefinitely.

Inference price is also not workflow cost. A reliable system may require data access, retrieval, tools, orchestration, evaluation, security, human review, and recovery from failure. A cheap model call can sit inside an expensive operating process.

Economist Joshua Gans offers a useful discipline for thinking about that difference. He treats AI prediction as an input to a decision and examines its value alongside substitutes, complements, automation, and system effects [3]. The point for an operator is straightforward: making one input cheaper does not make the complete decision cheap. The firm still needs data, judgment, and the capacity to act.

Modern generative systems stretch the prediction framing because they can produce artifacts and, when connected to tools, execute actions. The economic discipline still holds. As execution becomes cheaper, its complements become more important. A company needs to specify who may commit resources, what

constraints cannot be bypassed, what evidence must return, and how a bad action can be stopped or reversed. These are not features of the model. They are features of the operating system around it.

Capability has expanded as well. METR’s task-horizon research shows agents completing increasingly long, well-specified tasks in software engineering, machine learning, and cybersecurity [4]. But the researchers warn against turning this into a claim about whole jobs. Their benchmark is concentrated in technical tasks with clear scoring and relatively little organizational context. A 50 per cent success rate is not dependable operation. Messy work, tacit knowledge, competing goals, and institutional consequences change the problem.

The difference between adoption and autonomy is equally important. Stanford’s 2026 AI Index reports AI use in 88 per cent of surveyed organizations, while deployment of agents remains in the single digits across nearly all business functions [5]. The survey does not show that 88 per cent of companies have redesigned work or captured value. It shows that contact with AI is broad while delegated execution remains early.

These findings justify attention. They do not justify declaring the autonomous company inevitable.

1.2 Productivity belongs to a system

“Does AI improve productivity?” is usually too broad a question. Productivity is not an inherent property of a model. It is a result produced by a worker, a task, a tool, a process, and a measure.

In a staggered rollout involving 5,172 customer-support agents, Erik Brynjolfsson, Danielle Li, and Lindsey Raymond found that access to a generative assistant increased issues resolved per hour by 15 per cent on average. The gain was much larger for novice and lower-skilled workers and small for the most experienced workers [6]. That is strong evidence for a specific assisted workflow. It is not evidence that every function becomes 15 per cent more productive, or that the assistant can own the customer relationship.

A randomized study of experienced open-source developers reached a different result. Sixteen developers completed 246 real tasks in repositories they knew well. With early-2025 AI tools, they took 19 per cent longer, even though they believed the tools had made them faster [7]. The sample was small and the result belongs to those developers, repositories, tasks, and tools. It should not become a general claim that coding agents slow engineers.

Nor should it be dismissed because tools improved later. When METR tried to update the estimate in 2026, participation choices, task selection, non-compliance, and concurrent agent work made the treatment effect difficult to identify. The researchers thought newer systems probably offered more acceleration, but described the evidence for the magnitude as weak [8]. The methodological difficulty is itself instructive: once people delegate, supervise several runs, or choose different work because an agent is available, keyboard time stops being an adequate measure.

The support and software studies do not cancel each other. They describe different systems. One offered recommendations inside a high-volume workflow with observable outcomes. The other involved experienced maintainers changing mature codebases where understanding, integration, and review are substantial parts of the work. A person may become faster at a task while the system around that person remains unchanged—or becomes slower under the added load.

DORA’s 2025 research on software delivery describes AI as an amplifier of the surrounding organization: strong systems can convert local acceleration into performance, while weak ones can produce instability faster [9]. That finding is observational, not proof of a universal law. It nevertheless points toward the right unit of analysis. Faster generation alone does not tell us how much organizational value was created.

We need four different words:

Term	Meaning	A misleading proxy
Output	An artifact or action produced: code, analysis, copy, a recommendation, a transaction	Number of artifacts generated
Throughput	Accepted work that crosses the complete operating path	Tasks started or drafts produced
Outcome	A change in customer, operational, or strategic conditions	Work shipped without measured effect

Term	Meaning	A misleading proxy
Value	The benefit of that outcome after cost, risk, maintenance, and foregone alternatives	Model cost or hours reportedly saved

Cheap output is useful. It is not the same as throughput, outcome, or value. The difference matters to a COO because the company pays for the complete system while each team tends to report the part it produced.

1.3 The constraint moves

At any moment, end-to-end performance is limited by one or more constraints. Expanding capacity at one constraint is valuable until another limits the result. If a company can generate analyses, designs, code, and communications much faster without changing how it chooses, integrates, checks, and authorizes them, scarcity moves elsewhere.

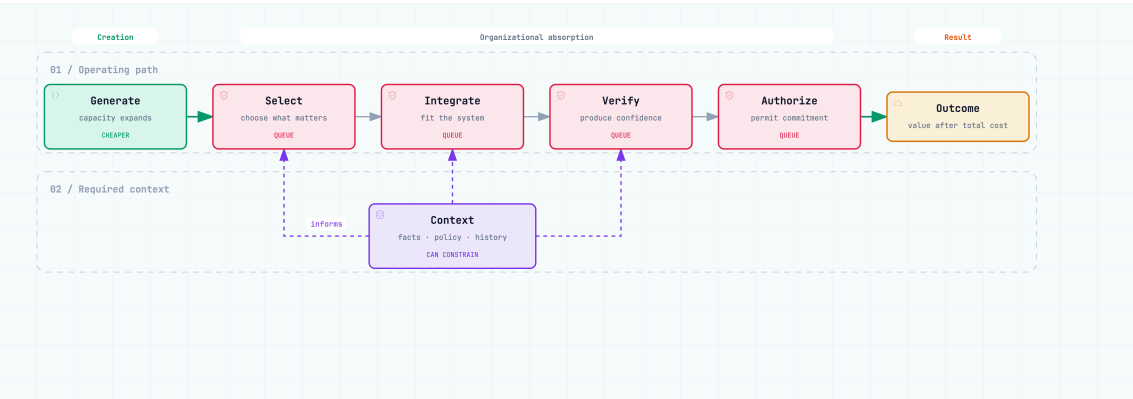


Figure 1.1: When generation capacity expands, the constraint often moves into selection, integration, verification, authorization, or context.

The exact sequence varies. Context can shape generation or arrive later to test an option. Verification may run continuously. Low-risk actions may be authorized in advance. The diagram is not a workflow template. It identifies five capacities that cheap production can expose.

None of these capacities is new. What changes is the imbalance between them. Generative systems can increase the arrival rate of work without increasing the capacity of teams that must absorb it. Agents can also participate in the downstream steps, changing who or what selects, checks, and acts. The constraint can therefore move faster and become harder to see in conventional activity measures.

Selection. Which request deserves work? Which option deserves commitment? Generation creates alternatives; it does not make the opportunity cost of choosing one disappear. A portfolio of cheap experiments can still consume the scarce attention needed to stop most of them.

Context. Does the work reflect the organization’s current intent, facts, constraints, history, and policy? A fluent answer built from stale commercial terms or the wrong customer state is fast rework. Supplying context is not a one-time prompt-writing exercise. Context must be current, accessible, permitted, and traceable.

Integration. Can the output enter a product, process, data environment, or customer relationship without transferring work to someone else? A generated feature that creates a new service dependency is not finished when its code runs. A proposal that cannot survive finance, security, or delivery constraints is not organizational throughput.

Verification. What evidence is proportionate to the consequence? A private brainstorming note and a payment instruction should not share an evaluation regime. Verification can include tests, source checks, simulations, peer review, monitoring, or comparison with a known baseline. It need not mean inspecting everything manually. It should explain why the system is trusted enough to act.

Authorization. Who—or what—may publish, deploy, spend, promise, reject, or bind the company? An agent may be technically capable of taking an action without having legitimate authority to take it. Authorization converts capability into bounded institutional power.

Accountability surrounds all five. It is not another box through which work passes. It answers a different question: who remains answerable for the outcome when work is divided among people, models, software, and vendors?

For the scaling COO, these queues often collapse into one calendar. Selection arrives as portfolio review. Missing context arrives as a request to recall why an earlier promise was made. Integration arrives as conflict between a business unit and a shared function. Verification arrives as a document that needs “one final look.” Authorization arrives as a message asking, “Are you comfortable if we proceed?”

The founder experiences these as unrelated interruptions. They are often one design failure: the organization depends on a person where it needs a decision system.

1.4 Cheap creation has an option cost

The apparent price of an additional option approaches zero long before its total cost does.

Return to the growing company. The fourth plan for the new business line may cost almost nothing to generate. It still asks the leadership team to compare assumptions, resolve conflicts with the existing business, test the economics, decide which capabilities remain shared, and state who may commit the company. Even a rejected plan can leave residue: forecasts, prototypes, provisional hires, technology choices, stakeholder expectations, and exceptions that somebody must later unwind.

A more honest cost equation is:

Total system cost = generation + selection + context + integration + verification + authorization + failure recovery + maintenance.

This is not an accounting standard. It is a checklist for costs that pilots often omit. If a pilot records only license fees and minutes saved, it can report a local gain while exporting cost to reviewers, platform teams, risk owners, or future maintainers.

The opposite mistake is to demand that every use case prove enterprise-wide value before it begins. Learning also has value. A small experiment can be justified by the uncertainty it resolves. But that requires an explicit learning question and a stop decision. “We can generate it” is not a reason to keep it.

1.5 Won’t agents absorb the new queues?

They will absorb parts of them. Agents can rank alternatives, retrieve context, open pull requests, run evaluations, and act within pre-approved limits. As capability improves, the constraint will move again.

That does not make operating design temporary. It makes operating design continuous.

An agent selecting investments still needs an objective and evidence of decision quality. An agent integrating code still needs access boundaries, tests, and a recoverable deployment path. An agent approving a refund has been given authority that must have a source, a limit, and a record. More capable execution changes where humans intervene; it does not remove the need to decide how intent, authority, evidence, and learning move through the company.

The claim also has boundaries. It is strongest in digital, decomposable, repeated work with observable results. Physical constraints may dominate manufacturing or logistics. Trust and relationship may dominate negotiation or care. Ambiguous strategy cannot be made sound merely by generating more strategic options. Stable processes should not be made fluid for the sake of appearing modern.

The point is not that production no longer matters. It is that production can no longer be assumed to be the limiting step.

1.6 What to do on Monday

Choose one recurring decision that still returns to the founder or COO. Use a real decision: a price exception, senior hire, product commitment, capital request, customer promise, or dispute between a business unit and a shared function.

1. Name the decision and the business outcome it is meant to protect. Do not name a meeting or document.
2. Write down the formal owner, then the person who makes the decision in practice.
3. Trace where the decision waits: selection, context, integration, verification, or authorization.
4. Record what the COO knows, checks, or weighs that the stated owner cannot yet access or apply.

5. Design one delegation test for the next occurrence: a clear intent, decision boundary, evidence requirement, and escalation condition. Do not silently retake the decision when the answer differs from your preference.

This exercise will often reveal a paradox: the organization is producing more while becoming harder to steer. Decisions multiply. Dependencies accumulate. Review work arrives without capacity. Exceptions live in messages and meetings. The COO remains indispensable not only because others lack skill, but because the company has stored context, judgment, and authority in one person.

That is not solved by telling the COO to let go. It is solved by making the missing operating structure explicit.

That liability is **Hidden Operational Debt**. It is where the next chapter begins.

2. Hidden Operational Debt

Most companies already have the evidence of how they really operate. It sits in customer conversations, CRM histories, issue trackers, approval threads, incident logs, calendars, financial systems, and the software itself.

Management rarely sees that evidence as a connected record. It receives a monthly summary. The summary says revenue is on plan, delivery is broadly on time, and customer satisfaction is stable. It does not show that pricing decisions now cross four desks, that one product manager resolves half the conflicts between two teams, or that the same customer problem has returned under six different labels.

Then the COO asks an AI assistant what is going wrong. The assistant reads a handful of documents and produces a familiar verdict: clarify ownership, improve communication, streamline processes.

The verdict is not false. It is almost useless.

A manager no longer needs a chapter to learn that organizations accumulate debt. Nor does a manager need six new labels for familiar dysfunction. General-purpose AI can propose a classification of the symptoms in seconds. The useful question is harder: **what evidence would allow the company to see the liability forming, estimate its effect, and intervene before another layer of management is added to absorb it?**

That is where Hidden Operational Debt becomes more than a metaphor.

2.1 The company is leaving a trail

Joshua Gans frames the economic effect of AI primarily through prediction and its contribution to decision efficiency [3]. Inside a company, however, a better prediction has to travel through an operating path. Someone must trust the inputs, understand the tradeoff, possess the right to decide, execute the decision, and observe what happened. A cheaper prediction has little value when that path is broken.

Hidden Operational Debt is the accumulated liability in those broken paths. It appears when the way the company is expected to operate and the way work actually moves have drifted apart.

The monthly report records the eventual output. The operational trail records the struggle:

- when a request entered the system;
- where it waited;
- which information was missing;
- how many people touched it;
- which rule or precedent was used;
- when a human overrode an automated recommendation;
- whether the decision was reversed;
- what the customer experienced;
- and whether the economic result matched the original assumption.

These events used to be expensive to assemble. Much of the trail was unstructured, so diagnosis depended on interviews, workshops, process maps, and the memory of experienced operators. Those methods still have value, especially when the record is incomplete. But they are no longer the only option. Language models can read thousands of tickets or decision threads, recover recurring themes, connect similar exceptions expressed in different words, and draft a first map of where the operating path keeps breaking.

This changes the economics of organizational diagnosis. It does not make the diagnosis automatic. The model can misclassify an exception, miss political context, or confuse correlation with cause. Its advantage is breadth and speed. It gives management a way to inspect actual workflows without commissioning a three-month study.

The source events must remain attached. A plausible summary without traceable evidence is merely a faster opinion.

2.2 Debt has an observable fingerprint

Six forms of debt are useful here, but as search lenses rather than a maturity model. Each leaves a different fingerprint in the data. Each is also affected differently when AI increases the volume and speed of work.

Debt	Signals already present in the business	What AI tends to amplify
Decision	Queue time, repeated approvals, executive escalations, overrides, reversals	More recommendations reach the same unresolved decision boundary
Context	Missing fields, conflicting sources, stale policies, time spent reconstructing history	Fluent answers spread weak or outdated context faster
Coordination	Handoffs, blocked work, re-opened tasks, dependency chasing, recurring alignment meetings	More work starts in parallel and arrives at shared dependencies sooner
Integration	Manual re-entry, reconciliation, failed syncs, bespoke handoffs, local data copies	Agents scale brittle interfaces and produce workarounds faster than they are retired
Knowledge	Repeated questions, rediscovered failures, undocumented rationale, inconsistent onboarding	Generated content grows while reusable judgment remains scarce
Control	Permission exceptions, missing evidence, after-the-fact approvals, unowned incidents	Action volume and access expand faster than review capacity

This is not a scoring system. One broken path can leave several fingerprints. A contract that waits five days may involve missing margin data, unclear discount authority, and a manual transfer between the CRM and finance model. Forcing it into one category would make the analysis worse.

The categories matter because the remedies differ. Decision debt is not repaired by a better summary if nobody has authority. Context debt is not repaired by delegation if the delegated owner cannot trust the numbers. Coordination debt is not repaired by another integration when the teams have incompatible priorities.

AI also changes the impact profile. In a manual organization, ambiguity limits throughput: only so much work can reach the unresolved boundary. Once agents can prepare analyses, write code, contact suppliers, or launch campaigns, far more work can arrive there. The debt may look smaller per task while becoming larger in aggregate.

The 2025 DORA research describes AI in software development as an amplifier of the surrounding organizational system rather than an independent performance intervention [9]. For operating-model design, the working hypothesis is that the same mechanism travels beyond software. A clear operating path becomes faster. A confused one produces confusion at machine speed.

2.3 Real practice: agent builders trace the path

The clearest public operating practices come from teams building agent products. They cannot manage a probabilistic system by reading its final answer alone. They retain the trace: the input, retrieved context, intermediate actions, tool calls, handoffs, errors, human feedback, and final state.

Anthropic's guidance makes a useful distinction between a transcript and an outcome. An agent can state that it booked a flight; the outcome is whether a valid reservation exists in the database. Its evaluation practice combines traces with outcome checks and tracks measures such as latency, token use, cost per task, and error rate. It also recommends sourcing new tests from failures seen in real use rather than relying only on synthetic examples [10].

That distinction belongs in the operating model of the company. A team can report that a customer issue was resolved; the outcome may be a repeat contact two days later. A business-unit lead can report

that a pricing decision was delegated; the trace may show that the COO rewrote the recommendation in a private message. A product team can close a feature request; the outcome may be lower activation because the underlying customer problem was misunderstood.

Customer-service systems show the same pattern in a narrower domain. Intercom’s current reporting for its Fin agent does not stop at how often the agent was used. It separates involvement, resolution, escalation, customer experience, and the performance of the content sources used to answer customers. The underlying conversation record includes the outcome state, rating, escalation reason, and content sources [11]. Those measures make it possible to ask whether automation produced a result, where it handed work back to a person, and which knowledge failed.

Agent observability tools take the approach further. Production conversations can be grouped into recurring categories while retaining links to the underlying traces. Error rate, latency, cost, evaluator feedback, and extracted attributes can then be compared by category and over time [12]. The point is not to endorse a particular tool. It is to notice the operating discipline: **keep the path, connect it to the outcome, and make recurring exceptions visible.**

Companies can apply the same discipline outside an AI product. A pricing approval, supplier exception, customer onboarding, incident response, or new-business launch is also a trace. It has a trigger, context, steps, decisions, exceptions, actors, and an outcome. Once that record is available, AI can help interpret it continuously.

2.4 The dashboard should show strain, not activity

Most executive dashboards describe results after the operating system has absorbed the cost. Revenue arrived. Tickets closed. Features shipped. Hiring completed. The company may be hitting those numbers through escalation, rework, and executive intervention that will not survive another doubling in volume.

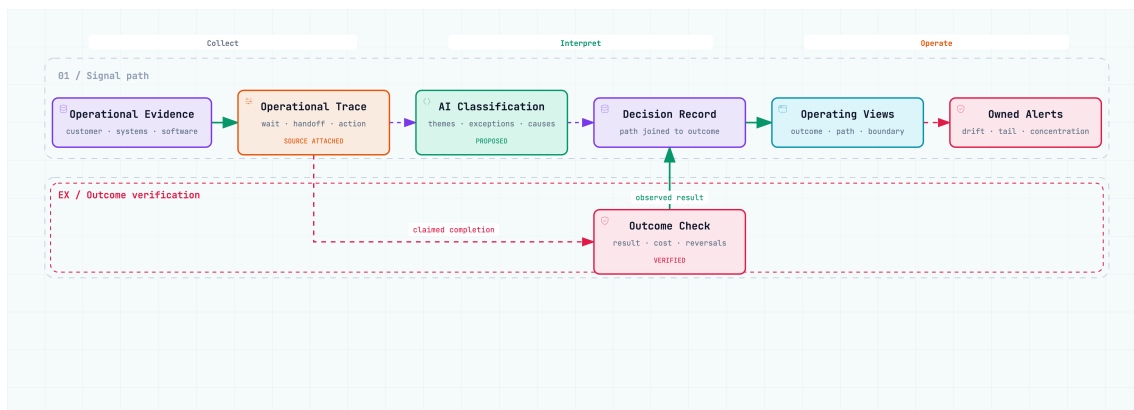


Figure 2.1: Operational evidence becomes useful when AI-supported interpretation remains tied to the trace, the observed outcome, and an owner who can respond.

An operational-debt dashboard sits beneath the result. It should let a leader move through three views.

The first is the **outcome**. Did the customer’s problem stay resolved? Did the deal produce the expected margin? Did the release improve the intended behavior? Did the new business line consume the shared capacity assumed in its model? A proxy such as “task completed” is not enough when the real result can be checked.

The second is the **path**. How long did the work wait compared with the time spent doing it? How many handoffs occurred? How often was an item reopened, reversed, or escalated? Where did the path diverge from policy? Which step repeatedly lacked context?

The third is the **system boundary**. What did people decide, what did agents execute, and where did control move between them? Useful measures include autonomous completion without correction, human intervention, policy exceptions, failed tool calls, missing evidence, and cost per successful outcome. Adoption is not a result. A high rate of AI use can coexist with poor economics and more rework.

No universal set of thresholds exists. A safety-critical approval and a low-value customer refund should not have the same path. Medians can also hide the cases that consume executive attention, so distributions matter. A COO often learns more from the slowest ten percent of consequential decisions than from average decision time.

The dashboard therefore needs drill-down, not decorative precision. Every aggregate should open into the events behind it. If escalations rise, management should be able to see whether they cluster around one rule, customer type, business unit, data source, or agent action. If the system cannot provide examples, the chart is not ready to govern anything.

2.5 Illustrative example: a new business line seen through its traces

Consider a company moving from one activity to several. The founder-COO launches the second business line personally, then appoints a business-unit lead and hands delivery to a Head of Operations. The organization chart says responsibility moved. Pricing behavior says otherwise.

The usual dashboard shows pipeline, bookings, gross margin, and perhaps the number of deals awaiting approval. It does not explain why pricing keeps returning to the founder.

The company already has most of the required evidence. The CRM holds the proposed price, customer segment, dates, owner, and deal result. Finance holds the cost assumptions. The contract system records non-standard terms. Product and operations systems expose capacity constraints. Email or messaging threads contain the exception rationale and the moment the COO became involved.

An AI analysis can turn the unstructured parts into proposed events: exception reason, missing context, policy cited, people consulted, decision made, and rationale. Those fields should be sampled and corrected by the operators who understand the process before they become metrics. Once calibrated, the analysis can run across the full period.

The resulting view might show:

- pricing-exception volume and rate by business unit and customer segment;
- median and tail decision time, separated into working time and waiting time;
- number of handoffs and requests for missing information;
- share of exceptions escalated to the COO, with the reasons for escalation;
- concentration of decisions by person, rule, and shared dependency;
- margin at approval compared with realized margin;
- reversals, contract amendments, and repeat exceptions for the same cause;
- deals lost or delayed while an exception remained open.

Now management can distinguish several stories that previously sounded identical. If most delays occur because shared-asset cost is unavailable, transferring pricing authority will not solve the problem. If the data is present but the business-unit lead escalates every deviation from the founder's preference, the liability sits in decision rights or behavior. If an agent proposes discounts inside the agreed envelope but humans repeatedly override them without recording why, the company has created an unobservable control loop.

Alerts should emerge from those stories. The system might flag a new cluster of exception reasons, a material change in tail decision time, repeated use of a stale cost model, an unusual rise in reversals, or growing concentration of approvals around one executive. The threshold must be agreed by the owner and calibrated against the company's own baseline. "Red" should mean that someone knows what to inspect and has the authority to respond.

This dashboard does more than save the COO time. It makes delegation falsifiable. The company can see whether decisions genuinely moved, whether outcomes remained acceptable, and where the next constraint formed.

2.6 Ask AI to build the first version

The first operational-debt view does not require a transformation program. It needs a consequential workflow, enough history to reveal recurrence, and access to an approved AI environment. Sensitive customer, employee, financial, and contractual data should be minimized or anonymized, and only placed in a system the company permits for that information.

Choose one path that is already causing concern: pricing exceptions, customer onboarding, incident resolution, supplier approval, or product requests. Export the last several months of records from the systems involved. Include timestamps and outcomes, not only narrative descriptions. Add the current policy, decision-rights document, or operating rule if one exists.

Then give Claude, ChatGPT, or another capable model a brief like this:

Act as an operating-system analyst. Analyze the attached, anonymized records for one workflow. Reconstruct the common path from trigger to outcome. Identify recurring waiting, handoffs, missing context, escalations, reversals, exceptions, manual reconciliation, and control failures.

For each pattern, separate: (1) what is directly observed in the records, (2) what you infer, and (3) what data is missing. Link every finding to representative source records. Do not rank employees or treat activity volume as performance.

Produce four outputs: a proposed event schema; a ranked set of operational-debt hypotheses; a dashboard specification with metric definition, source, owner, and drill-down; and alert rules calibrated to the observed baseline. Connect path measures to customer, quality, or economic outcomes wherever the data permits. State clearly where no causal conclusion is possible.

The first response will expose data gaps and bad definitions. That is useful. Ask the model to generate the transformation logic, queries, or formulas only after the operating owner confirms that the proposed events mean what the model thinks they mean.

Then make the build request explicit:

Using only the validated fields, generate a working local dashboard or an implementation specification for our existing BI tool. Create three views: outcome, operating path, and human-agent boundary. For every metric, show the formula, data source, refresh cadence, owner, and links to the underlying records. Add alert logic for changes against our baseline. Do not invent unavailable fields; list them as instrumentation gaps.

The exact output may be an HTML prototype, SQL and chart definitions, or a specification for Power BI, Looker, Metabase, or the company's existing tool. There is no reason to begin with a new platform.

Before relying on the result, take a sample from each major pattern and compare the AI classification with the source material. Keep the corrections. They become examples for the next run and the beginning of an evaluation set. When a new failure appears, add it. This is the same feedback loop used by teams that improve production agents: real traces become tests, tests protect the definition, and the definition makes the dashboard more trustworthy.

The output should not be a one-time diagnostic deck. Schedule the analysis at a cadence appropriate to the workflow. Let the system surface new clusters and threshold breaches. Let the owner inspect the underlying cases, decide whether a liability is real, and record the intervention. Then watch whether the path and outcome change.

That final step matters. A dashboard that detects recurring debt but cannot trigger a decision becomes another source of operational debt.

2.7 From reporting to an operating instrument

Operational visibility can easily become surveillance. Measuring handoffs and delays at individual level invites gaming and punishes the people who absorb the hardest exceptions. The unit of analysis should normally be the workflow, decision boundary, policy, interface, or recurring condition. Individuals appear when they own the remedy, not as targets on a leaderboard.

AI-generated categories can also harden a weak interpretation. Preserve samples, let operators challenge classifications, and show confidence and missing data. When financial or customer consequences are uncertain, say so. The aim is not a magical debt score. It is a shorter path from signal to investigation to operating change.

Once that path exists, Hidden Operational Debt stops being an annual workshop topic. It becomes observable state. Customer conversations reveal where policy and product diverge. Software logs reveal where work repeatedly fails or returns. Decision traces reveal where delegation is nominal. Agent traces reveal where autonomy exceeds control or where humans are still paying invisible interest.

The company can then ask a better question than "Where should we use AI?"

It can ask: **Where is our operating system preventing intelligence - human or artificial - from becoming a sound decision and a verified result?**

That question leads to the boundary between adding an AI tool and changing how the company operates. It is the subject of the next chapter.

3. Why Tools Do Not Change the Operating Model

The easiest AI pilot to approve is attached to a task one person already owns. Draft the email. Summarize the meeting. Produce the first analysis. Write the test. The user can adopt the tool without asking anybody else to change.

That is why early gains appear first in individual work. It is also why they can mislead management about the company.

A randomized field experiment gave half of 6,000 knowledge workers access to generative AI inside their normal office applications. Workers who used the tool spent less time on email and appeared to complete documents faster. Meeting time did not change significantly. The authors' explanation is revealing: behavior that one person could change moved before behavior requiring coordination with others [13].

The tool improved a local action. The shared routine remained intact.

That distinction explains why adoption dashboards and operating results often diverge. A company may report thousands of prompts, broad license activation, and enthusiastic employee stories while decisions still wait for the same executives. The evidence of use is real. The inference about transformation is not.

Tools spread through tasks because the user can usually decide alone. Operating models change through agreements among people who own different constraints. Sales cannot unilaterally redefine recognized margin. A business unit cannot promise shared engineering capacity that the platform team has not allocated. An agent cannot give itself permission to change a customer contract. These dependencies make system change slower than tool adoption even when everybody likes the tool.

3.1 What an operating model contains

An operating model is the set of arrangements that converts strategy into repeated results. It includes more than process. It determines:

- what the company is trying to achieve and what takes priority;
- what information is authoritative;
- where decisions sit and how they escalate;
- how work crosses teams, systems, and business units;
- which controls bind action;
- what evidence is retained;
- and how results change the next decision.

An AI tool changes the operating model only when it changes one or more of those arrangements. A meeting summary may save time. A decision record that automatically captures the evidence, owner, authority, dissent, and later outcome changes how the company learns. The interface looks similar; the institutional effect is different.

The definition also prevents an org-chart mistake. Reporting lines are one part of the model, but moving a team under a different executive does not repair a missing interface or an ambiguous right. Companies often reorganize because boxes are visible while decision paths are not. Six months later, the same conflict has new attendees.

Consider a discount approval. Sales prepares the request, finance checks margin, legal checks terms, and the COO approves. An assistant can draft each handoff. The request moves through the same rights, queues, and data gaps, only with better prose. If the company instead defines a discount envelope, makes current cost and capacity visible, delegates decisions inside the envelope, and logs exceptions with outcomes, the operating model has changed. AI may support the new path. It did not cause the redesign.

3.2 The paved cow path

Automation often preserves an old compromise because the current process is the easiest specification available. The team maps the steps, connects the systems, and asks an agent to move work between them. Nobody asks why the steps exist.

This is the paved cow path failure. A temporary approval becomes a permanent API. A spreadsheet built to reconcile two unreliable systems becomes an automated reconciliation service. A weekly status meeting becomes an AI-generated status pack. Friction declines, so the underlying liability attracts less attention.

The result can be rational. Paying the interest more cheaply may be worthwhile while the company prepares a deeper change. The error is reporting the saving as transformation.

Consider customer onboarding. A new customer completes a form. Operations re-enters information into an internal system. Finance checks commercial terms. Security reviews the configuration. Customer success explains missing details to the customer. An agent can read the form, populate the internal record, draft the requests, and chase replies. Cycle time falls.

If the same data remains defined differently across systems, the agent has become an unusually efficient reconciler. If every non-standard term still waits for a senior executive because authority is unclear, the agent has become an unusually efficient escalator. Both uses may create value. Neither changes the company's ability to absorb a new product or business unit.

A redesign would ask which customer facts should be captured once, which system owns them, which configurations fit a standard policy, which exceptions require judgment, and who may decide them. The agent then operates inside a simpler path and makes that path observable. Automation follows design rather than substituting for it.

There are three levels of improvement:

Level	What changes	Typical result
Task	One person produces an artifact faster	Local time saving
Workflow	Several steps, handoffs, or systems are redesigned	End-to-end throughput or quality improves
Operating model	Intent, rights, interfaces, controls, evidence, or learning change	The company can behave differently at scale

The levels build on one another. A task improvement can fund a workflow redesign. A successful workflow can reveal a better allocation of authority. Problems begin when management assumes the levels are interchangeable.

There is no shame in staying at the first level. A lawyer who drafts a first version faster or an analyst who explores more alternatives may be worth the license. The organization should name the result accurately. Honest local gains are more useful than inflated enterprise claims because they reveal where the next constraint actually moved.

3.3 Local output can create system load

Every team can optimize itself and make the company slower.

Marketing generates more campaigns, increasing review and sales follow-up. Product produces more specifications, increasing engineering selection work. Coding agents create more pull requests, increasing integration and security review. Finance generates more scenarios, increasing the number of choices leadership must resolve. The local dashboard is green because output rose. The system queue grows elsewhere.

The correct unit of analysis is the complete path from intent to verified result. Ask where work waits, where it returns, where a human reconstructs context, and where exceptions concentrate. If the gain disappears when downstream review, rework, failure recovery, and maintenance are included, the pilot has moved cost rather than removed it.

This does not require a central transformation office to approve every use. Teams should experiment. But every material pilot needs an owner for the end-to-end result, not merely the tool. That owner should know which metric may improve, which downstream cost may rise, and what would justify stopping.

The owner also needs a counter-metric. If generated sales proposals increase, track acceptance, margin, revision, and delivery exceptions. If code output rises, track accepted throughput, change failure, review load, and maintainability. If an assistant closes support cases, track repeat contact and customer outcome. A local measure tells the team whether the tool is active. A counter-metric shows whether the rest of the system is paying for that activity.

3.4 Bottom-up adoption still matters

Operating design should not become an excuse for central control. Many valuable uses are discovered by people close to the work. A mandated enterprise program can freeze learning while it waits for a perfect architecture.

The better pattern has two speeds. People experiment with bounded, reversible work using approved tools and data. When a use begins to affect customers, shared systems, money, rights, or other teams, it crosses an operating threshold. The owner must then make context, authority, controls, evidence, and downstream effects explicit.

This threshold avoids both extremes. The company does not review every private draft. It also does not allow a useful personal shortcut to become an invisible production dependency.

3.5 The test

Before calling an initiative an operating-model change, complete this sentence:

We used to rely on **[old intent, context, right, interface, control, evidence, or learning mechanism]**. We now rely on **[new mechanism]**, which allows **[specific changed behavior]** while protecting **[specific consequence]**.

“Employees now use copilots” fails the test. “Business-unit leads now decide discounts inside a live margin and capacity envelope, while exceptions route to finance with a trace” passes it.

An AI-using leader can apply the test directly. Give an approved assistant the pilot description, workflow records, policy, and outcome data. Ask it to produce two maps: what changed inside the task, and what changed in the surrounding operating system. Require source evidence for each claim and ask which costs may have moved to another team.

Pilot audit prompt: Separate the observed change into task, workflow, and operating-model effects. For each claimed benefit, identify the complete path to a customer or economic outcome, the downstream work created, the decision right affected, the evidence available, and the strongest alternative explanation. Do not treat adoption, output volume, or reported time saved as proof of enterprise value.

The response is an argument to test, not a verdict. Its value is that management can perform this challenge while the pilot is still small.

The distinction protects the company from two mistakes. The first is underestimating a useful local tool because it did not transform the enterprise. The second is overestimating adoption because many people opened an application.

ATOM begins where the tool ends. It describes the arrangements around intelligence: how intent enters, how context is governed, how decisions receive authority, how humans and agents execute, what evidence returns, and how the company learns.

II

Part II: The ATOM Model

4	ATOM: Intent In, Evidence Out	21
4.1	Intent is a usable constraint	22
4.2	Context is governed state	22
4.3	A decision is a commitment	22
4.4	Execution is a designed combination	22
4.5	Evidence is an output	22
4.6	Learning has an owner	23
4.7	One refund through the loop	23
4.8	What ATOM is not	23
5	Context Is Infrastructure	25
5.1	The context envelope	25
5.2	Four tests before information enters the envelope	26
5.3	Separate memory by purpose	27
5.4	Make conflict visible	27
5.5	Untrusted content is not instruction	27
5.6	Build context as a product	28
5.7	What to build this week	28
6	Design the Decision Before the Agent	29
6.1	Inventory commitments, not meetings	29
6.2	A decision has more than an answer	29
6.3	Consequence and reversibility shape autonomy	30
6.4	Authority is specific	31
6.5	Confidence does not equal authority	32
6.6	Design the escalation, too	32
6.7	An illustrative pricing decision	32
6.8	Turn a recurring escalation into a specification	32
7	Human-Agent Execution	34
7.1	Choose the right executor	34
7.2	Decompose the work without fragmenting the outcome	34
7.3	Design the handoff	35
7.4	The person needs a control surface	36
7.5	The Synthesizer	36
7.6	Multi-agent is an architecture choice	37
7.7	Learn the boundary before raising autonomy	37
7.8	Map one live loop	37

4. ATOM: Intent In, Evidence Out

An operating model should be drawable without a slide deck.

ATOM has six connected layers and one control plane. Read them clockwise as a loop:

1. **Intent** states the outcome, priority, boundaries, and time horizon.
2. **Context** supplies the authorized facts, policy, history, and current state.
3. **Decisions** convert intent and context into a committed choice.
4. **Execution** combines people, deterministic software, workflows, and agents.
5. **Evidence** records what happened and what result followed.
6. **Learning** changes the relevant assumption, context, policy, capability, or strategy.

The **control plane** crosses the loop. It provides identity, permissions, limits, approvals, monitoring, intervention, rollback, and accountability.

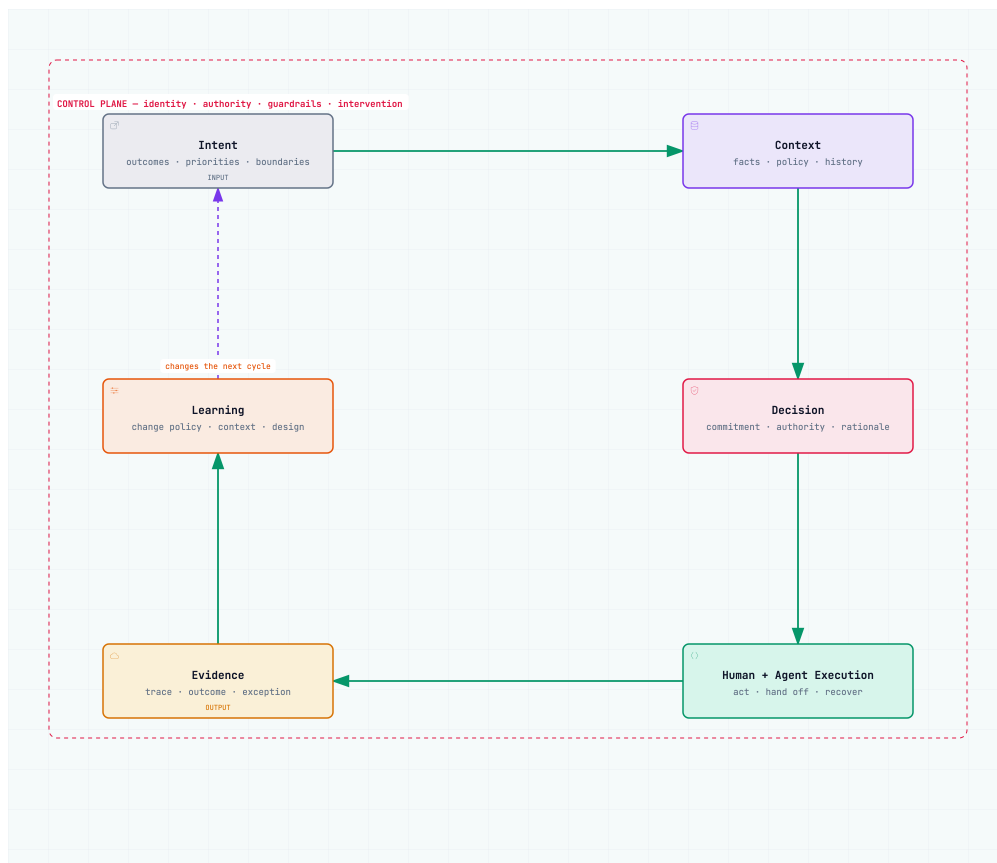


Figure 4.1: The canonical ATOM loop: intent enters, evidence returns, and learning changes the next cycle. The control plane constrains the whole loop.

That is the complete model. The rest of the book makes each part operational.

The loop can be entered from two directions. Leadership can begin with intent: a strategic outcome that needs an operating path. Operations can begin with evidence: a recurring failure, customer signal, or exception that reveals the present system. In both cases, ATOM asks the company to connect the whole cycle. Strategy without returning evidence becomes instruction. Evidence without a route to authority becomes reporting.

4.1 Intent is a usable constraint

“Grow efficiently” is not intent. It does not tell a team what to trade away, which customer matters, or how long the choice should hold.

Usable intent names an outcome and its boundary. A new business unit might be asked to prove that a particular customer problem can support a positive contribution margin without consuming more than a stated amount of shared engineering capacity. The unit now has room to act and a reason to refuse attractive work that violates the experiment.

Intent is not a cascade of slogans. It is a constraint that makes local decisions coherent.

A useful intent statement contains tension. “Increase adoption” invites activity. “Increase activation among mid-market customers without raising implementation effort per account” names a result and protects an operating condition. Teams can now reject a feature that attracts sign-ups but increases manual setup. An agent can search for options without being allowed to redefine the tradeoff.

Intent also has a shelf life. A temporary cash constraint should not remain embedded in agent policy after financing changes. A market-entry priority should not silently govern a mature business. The owner and review date belong beside the statement.

4.2 Context is governed state

Context includes live business data, policy, customer history, commitments, prior decisions, and the state of the work. It also includes provenance: where the information came from, when it was updated, who owns it, and who may see it.

More context is not always better. An agent handling a refund does not need the entire customer data lake. It needs the current account state, the applicable policy, relevant interaction history, and authority limits. Excess context raises cost, creates privacy exposure, and makes conflicts harder to detect.

Context is where many implementations hide organizational choices inside technical retrieval. Selecting an “authoritative” document decides which function’s view governs. Setting freshness decides how long the company tolerates drift. Granting access decides which actor can combine sensitive facts. These are operating decisions implemented through infrastructure.

4.3 A decision is a commitment

Analysis becomes a decision when an authorized actor commits the company. The actor may be a person, a deterministic rule, or an agent acting within delegated bounds. ATOM does not assume the human always makes the final call. It requires the source and limit of authority to be explicit.

Every recurring decision should expose its owner, inputs, criteria, consequence, reversibility, evidence requirement, and escalation path. Without these, an agent may offer advice but cannot receive legitimate autonomy.

Decision design also clarifies the role of the center. The center should not decide every case shared functions care about. It should define the enterprise constraints, resolve conflicts among units, and own decisions whose consequences cross the whole company. Local owners decide inside that space.

4.4 Execution is a designed combination

Execution belongs to the system best suited to the work. Deterministic software handles stable rules. Models interpret ambiguity. Agents manage bounded multi-step work. People set intent, exercise judgment, negotiate conflict, absorb legitimate exceptions, and remain accountable.

The design question is not “human or AI?” It is where each hands work to the other, what state transfers, and what happens when the receiver cannot proceed.

Execution may run in parallel. Several agents can research, test, or prepare options at once. The operating path still needs a convergence point: a decision, accepted artifact, verified state, or stopped run. Parallelism without convergence creates more outputs than the company can integrate.

4.5 Evidence is an output

ATOM produces two outputs: a business result and evidence about how the result was produced. Evidence includes source versions, decisions, approvals, actions, tool responses, exceptions, cost, and outcome measures. Its depth should match the consequence.

An organization that only records the final answer cannot distinguish a lucky result from a reliable system. It also cannot investigate harm, improve an agent, or defend the decision later.

Evidence is selective, not total surveillance. The company does not need to retain every internal token or employee action. It needs the material facts required to reconstruct a consequential result. Retention, access, and deletion are themselves control decisions.

4.6 Learning has an owner

Learning is not a generic retrospective. Evidence may suggest that the model needs a new evaluation, the context source needs repair, a permission is too broad, a decision threshold is wrong, or the strategy itself failed. These changes belong to different owners. Letting one agent rewrite all of them would collapse learning and control into the same authority.

Approved and reusable changes enter the **ATOM Playbook**: the maintained collection of decision specifications, unit contracts, context products, controls, evaluation cases, and operating lessons used in future loops. It is curated memory, not a transcript archive.

ATOM closes the loop by sending approved changes to intent or context. It does not let every observation alter production behavior immediately.

Approved reusable knowledge enters the **ATOM Playbook**: the maintained set of intent statements, context definitions, decision specifications, unit contracts, controls, evaluation cases, and operating lessons that other loops can apply. The Playbook is not a transcript archive. It contains what the organization has chosen to remember.

4.7 One refund through the loop

Take a customer who requests a refund after a service failure.

Intent says the company will restore trust quickly while protecting against fraud and preserving consistent treatment. **Context** provides account state, transaction history, service incident, current refund policy, customer communications, and verified identity. **Decision** grants an agent authority to approve standard refunds below a threshold when the service failure and identity are confirmed. **Execution** issues the refund through a payment tool, informs the customer, and opens a service-quality record. **Evidence** retains the sources, policy version, decision, action result, and later repeat contact. **Learning** may reveal that one failure category is creating avoidable refunds and route that pattern to the product owner.

The control plane binds each step. The agent uses a scoped identity. It cannot change the policy or send money to a different account. A high amount, identity conflict, or vulnerable-customer flag routes to a person. The customer can challenge the result.

Now alter one condition. The payment system confirms success, but the account ledger fails to update. Execution is only partly complete. Evidence should make the split state visible and trigger recovery. A dashboard that records the agent's tool call as a successful refund would teach the organization the wrong lesson.

The example is small enough to draw and rich enough to expose the architecture. The same grammar can govern a capital allocation, production incident, hiring decision, or new-business experiment. The depth of context and control changes with consequence; the loop does not.

4.8 What ATOM is not

ATOM is not an autonomous company. Organizations contain power, conflict, law, identity, trust, and consequences that cannot be reduced to a feedback diagram.

It is not a technology stack. A company can implement the loop with ordinary databases, workflow tools, meetings, and carefully bounded assistants. New infrastructure is justified only when the current tools cannot provide the needed context, action, or evidence.

It is not a replacement for Lean, Agile, product operating models, or sound functional management. It inherits their concern for flow, feedback, customer value, small batches, and capable teams. It adds a more explicit architecture for decision rights, machine actors, evidence, and control.

ATOM also resists a weakness common to feedback metaphors: the assumption that the measured target is the right one. A tight loop can optimize the wrong outcome with great efficiency. Leadership remains responsible for choosing intent, listening to people the metric excludes, and stopping a locally successful system that damages the wider company.

Nor is ATOM the org chart. The same loop can operate inside a function, across a customer journey, or within a business unit. At scale, repeated loops can form semi-autonomous units, provided their interfaces and central constraints are clear.

The shortest definition is therefore the most useful:

ATOM is a governed learning loop that converts intent and authorized context into decisions, human-agent execution, and evidence.

A reader should now be able to draw it. Start with the six layers. Draw the return from learning. Place control across the path. If an implementation cannot say what enters, who may decide, what may act, what proof returns, and who may change the system, it is not yet an ATOM loop.

When mapping a real workflow, use the diagram as an interrogation device. Ask an approved AI assistant to infer the six layers from actual records, then mark every answer that is absent, contradictory, or merely assumed. The blank spaces usually matter more than the polished map. They show where the company is depending on private memory, informal authority, or unobserved recovery work.

5. Context Is Infrastructure

The first failure of an agent is often blamed on the model. Inside a company, the failure more often begins with the material the model was allowed to see.

Imagine a supplier-selection agent. It compares price, lead time, and defect history and recommends the incumbent. The answer is well reasoned. It is also wrong. The contract repository contains an old agreement, the current sanctions policy lives in a legal memo, and the operations system uses a different supplier identifier. A senior buyer notices the conflict because she remembers the negotiation.

Calling this a hallucination would let the company avoid the real issue. The agent acted on a context product the organization never designed.

Humans have always compensated for poor organizational context. They remember that the dashboard is late, know which spreadsheet contains the real forecast, call the person who negotiated the exception, and reinterpret the rule according to what happened last quarter. That hidden reconstruction is expensive, but it is also forgiving. An experienced operator can notice that two records which look identical are not the same customer.

Agents make context debt visible. They can retrieve more material and connect it faster, but they do not inherit the unwritten map of authority that lives in an operator's head. Giving them access to every repository does not solve this. It increases the number of plausible mistakes they can make.

5.1 The context envelope

ATOM gives each recurring decision or task a **context envelope**. It is the smallest body of current, authorized, attributable information required to do the work.

Part	Question
Task state	What is happening now, and what has already happened?
Enterprise facts	Which business data is required, and which source is authoritative?
Policy	Which rules, thresholds, and obligations apply?
History	Which prior decisions, commitments, and outcomes matter?
Provenance	Where did each item come from, who owns it, and how fresh is it?
Access	Who or what may retrieve, combine, retain, or disclose it?

The envelope is deliberately smaller than “company knowledge.” A pricing decision does not need the whole CRM, every call transcript, and the complete finance warehouse. It may need the customer's current segment, contracted terms, payment status, service cost, available capacity, relevant commercial policy, and the last approved exception. Everything else competes for attention, raises privacy exposure, or creates another possible conflict.

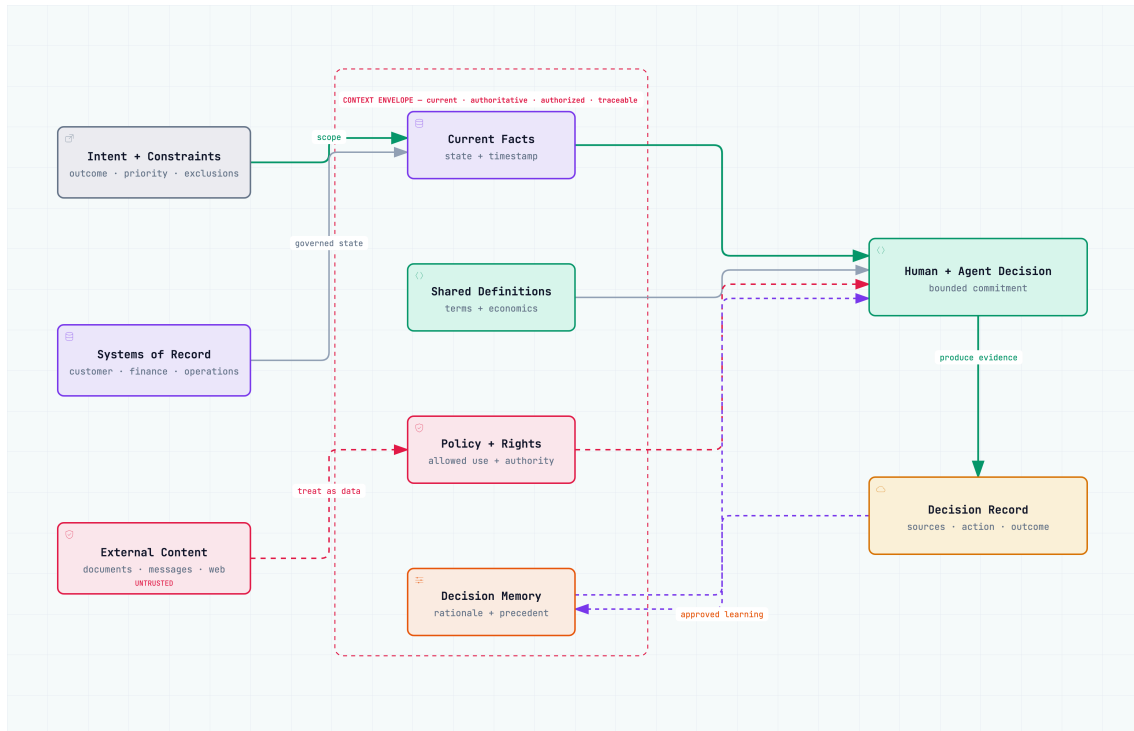


Figure 5.1: A context envelope admits governed facts, definitions, policy, and decision memory while treating external content as untrusted data.

This is not merely an architectural preference. Teams building long-running agents now describe context as a finite resource that must be curated. Anthropic’s engineering guidance distinguishes context engineering from prompt engineering: the job is to select and maintain the system instructions, tools, external data, history, and memory most likely to produce the desired behaviour. It also warns that simply increasing the amount of context can reduce focus and retrieval precision [14]. The organizational implication is more important than the model behaviour. Someone must decide what deserves to become context, under whose authority, for how long, and for which decision.

Retrieval is not truth. A search system can return an obsolete policy with perfect semantic relevance. A long conversation can contain a decision that was later reversed. A model can merge two customer records into one fluent account. Context has to be governed before it is generated into prose.

5.2 Four tests before information enters the envelope

Each item admitted to a consequential decision should survive four tests.

Relevance. Does this fact change the decision or the confidence required? “It might be useful” is not enough. Excess material hides the few facts that matter.

Authority. Is this the source that governs the issue? A slide describing the discount policy is not equivalent to the approved policy. A CRM note about a contractual promise is a signal to check the contract, not a substitute for it.

Freshness. Could the fact have changed since it was validated? Freshness is defined by the work. Yesterday’s warehouse capacity may be stale for dispatch but adequate for a quarterly footprint review.

Permission. Is this actor allowed to access and combine the information for this purpose? Access to two systems separately does not automatically authorize joining their data. Nor does the ability to read a record imply permission to retain it in agent memory.

These tests turn a retrieval pipeline into an operating contract. When one fails, the system should retrieve a better source, reduce its confidence, request clarification, or stop. It should not hide the gap behind fluent language.

5.3 Separate memory by purpose

Organizations often ask for an agent “with memory,” as if memory were a single feature. It is better to separate at least four kinds.

Working state lasts for the task: what has been tried, what remains open, and what each tool returned. It should make interrupted work resumable without turning a scratchpad into a permanent corporate record.

Relationship memory retains permitted facts about an account, supplier, employee, or counterparty. It needs ownership, correction, retention, and visibility rules. A useful customer preference may still be personal data.

Institutional memory preserves policies, precedents, decisions, and outcomes. It is not a document archive. It is a maintained account of what the organization has decided, why, and what superseded it.

Learning memory contains examples used to test or improve the system: successful cases, edge cases, overrides, incidents, and deliberately difficult scenarios. Its purpose is evaluation, not live decision-making.

The separation matters. A failed agent trace may belong in an evaluation set but not in the next customer’s prompt. A customer’s preference may belong in an account record but not in a general training corpus. A temporary exception may inform one task without silently becoming precedent.

Memory also needs a forgetting mechanism. Wrong facts that remain easy to retrieve acquire institutional weight. Expired exceptions become shadow policy. A corrected customer record survives in a conversation summary. The design question is therefore not only, “What should the agent remember?” It is, “What has the authority to make it forget?”

5.4 Make conflict visible

Every authoritative source used for consequential work should answer four questions: Who owns it? When was it last validated? What does it govern? What replaces it?

If two sources conflict, the agent should not silently choose the one with the clearer wording or later timestamp. It should apply a declared precedence rule or stop. If a source has expired, the system should surface that fact at the point of use. If a human supplies an exception, the exception should have scope, rationale, owner, and duration rather than becoming an invisible new rule.

The NIST Generative AI Profile treats governance, content provenance, data privacy, testing, and incident handling as lifecycle concerns rather than prompt-level features [15]. That framing is useful for operators. The quality of context depends on work that happens outside the model: defining records, assigning ownership, resolving contradictions, controlling access, and maintaining sources.

This can feel slower than connecting an agent to a drive and seeing it answer questions in an afternoon. The difference appears when the answer changes a price, blocks a shipment, commits capacity, or sends a customer message. At that point, a source map and a precedence rule are not administrative overhead. They are part of the product.

5.5 Untrusted content is not instruction

An agent may need to read customer emails, supplier documents, web pages, and support tickets. Those materials can contain requests and evidence. They can also contain instructions—malicious or accidental—that conflict with the organization’s intent.

The security problem is easy to miss because the same model reads both trusted directions and untrusted content. OpenAI describes prompt injection as a form of social engineering and recommends limiting access to the minimum data and capability required. Its more recent design guidance frames the dangerous combination as an untrusted **source** connected to a consequential **sink**, such as transmitting data or invoking a tool [16].

ATOM handles this in the envelope. Each source has a trust class. Retrieved content supplies facts; it does not acquire authority to change the task. Tools enforce permissions outside the prompt. A supplier PDF can state a new bank account, but it cannot authorize the payment system to use it. A customer message can request a refund, but it cannot alter the refund ceiling. The action still has to pass the decision specification and the control plane.

5.6 Build context as a product

A context product has users, an owner, a service expectation, and quality measures. The pricing context product for a business unit might join current cost, available capacity, standard terms, customer segment, outstanding commitments, and the discount policy. Finance owns cost definitions; operations owns capacity; commercial leadership owns the decision envelope. The context product brings those facts together without pretending one function owns the whole truth.

Its measures should expose whether the organization can supply usable state:

- source freshness and coverage;
- retrieval success for required fields;
- unresolved conflicts between authoritative sources;
- missing or ambiguous entity identifiers;
- permission denials and unusual access combinations;
- decisions delayed by manual context reconstruction;
- reversals caused by missing or stale context;
- and the proportion of outputs whose claims can be traced to a source.

These are not model benchmarks. They reveal the condition of the operating system around the model.

The product also needs a change path. When finance modifies the cost definition, who tests the pricing decisions affected by it? When a policy changes, which agents and human workflows receive the new version? When an operator discovers an important exception, how is it reviewed and either codified or allowed to expire? Without that maintenance loop, the initial integration becomes another source of debt.

5.7 What to build this week

Start with one recurring decision, not a company-wide knowledge graph. In an approved AI workspace, provide the blank decision template from the field kit and samples of the materials currently used by the people who make that decision. Where policy permits, add structured exports with sensitive fields removed or masked.

Ask the assistant to produce five outputs:

1. a source map showing each required fact, its current location, owner, and update frequency;
2. a conflict report listing duplicate definitions, stale documents, identifier mismatches, and unresolved precedence;
3. a proposed context envelope separating required, optional, and prohibited information;
4. a machine-readable source manifest with owner, freshness rule, trust class, and access scope;
5. ten test questions whose answers should fail when an essential source is missing, stale, or contradictory.

Do not ask the model to settle ownership disputes. Ask it to find them. Source owners then validate the map, decide precedence, and set failure behaviour. Connect the agent only after the envelope survives those tests.

An agent's autonomy cannot exceed the authority and quality of its context. Once context is treated as infrastructure, experienced judgment becomes easier to distribute. The company no longer depends on each operator remembering which part of the official system is unofficially wrong.

6. Design the Decision Before the Agent

Companies frequently delegate work without delegating the decision. A manager prepares the recommendation, collects the evidence, and coordinates the stakeholders. The founder still chooses. Installing an agent in that arrangement gives the manager faster preparation and the founder more decisions to absorb.

The founder may feel less overloaded at first because the briefs arrive cleanly. Soon they arrive more often. Cheaper analysis lowers the threshold for asking for analysis. Options multiply. Every team can generate a convincing case. The approval queue becomes the last scarce resource.

The remedy begins before the agent is selected. Design the decision.

6.1 Inventory commitments, not meetings

A decision inventory records moments when the company commits money, capacity, access, reputation, or a course of action. “Weekly pricing committee” is not a decision. “Approve a discount outside the standard envelope” is. “Product review” is not a decision. “Move three engineers from the core product to the new business for six weeks” is.

This distinction sounds simple until leaders attempt the inventory. Calendars show where discussion occurs, not where commitment happens. Many commitments are hidden inside process steps: a sales promise entered as a note, a hiring profile approved through silence, a delivery date accepted when nobody challenges it. Other meetings revisit decisions that were supposedly delegated.

For each recurring decision, record:

- the outcome it protects;
- the accountable owner;
- required inputs and their authoritative sources;
- criteria and prohibited conditions;
- consequence if wrong;
- reversibility and cost of reversal;
- time available;
- authority granted to people, rules, and agents;
- evidence retained;
- escalation and appeal;
- and the date the design itself will be reviewed.

The inventory often exposes phantom delegation. A business-unit lead owns the result but cannot approve price, hire within budget, or commit shared capacity. Accountability moved on the organization chart; authority stayed at the centre.

It can also expose orphaned decisions. Everyone contributes analysis, but nobody is clearly entitled to decide. The issue climbs until a senior leader absorbs it. What looks like a need for better collaboration is sometimes a missing owner.

6.2 A decision has more than an answer

An operational decision is a small system. It starts with a trigger, builds a state from evidence, evaluates alternatives, applies values and constraints, commits an action, and creates consequences that should be observed. The answer—approve, reject, price, route, hire, allocate—is only one event in that system.

This is why automating an answer can fail even when the answer is statistically good. The trigger may be wrong. The inputs may omit a promise. The action may exceed authority. Nobody may observe whether the result was useful. The company has improved prediction without improving the decision.

Joshua Gans’s microeconomic treatment of AI is valuable here because it separates prediction from the complementary parts of a decision. When prediction becomes cheaper, judgment, action, data, and the design of the surrounding system can become more valuable [3]. A model can estimate churn or delay. The company still decides which errors matter, what it owes the affected party, and which action is legitimate.

The practical unit of design is therefore not “an AI use case.” It is a recurring commitment with an owner and a feedback path.

6.3 Consequence and reversibility shape autonomy

Not all uncertainty deserves a human approval. A low-cost, reversible action can often be delegated with monitoring. A consequential but reversible action may be executed within a budget and reported immediately. An irreversible or rights-affecting decision usually deserves stronger review, separation of duties, and recourse.

Consequence	Reversibility	Default design
Low	Easy	Act within policy; log the result
Material	Easy	Act within limits; monitor and sample
Low	Difficult	Propose or require approval
High	Difficult	Named human decision, strong evidence, independent control

This is a starting posture, not a formula. Frequency changes exposure. Ten thousand individually trivial credits can become a material loss. A reversible public message may still damage trust. A recommendation may have no formal authority yet strongly influence an employee’s future. The owner has to consider aggregate value, affected parties, detectability, concentration, and the time available to correct a mistake.

Human review is also not automatically a strong control. A tired approver presented with a polished recommendation may click accept. If the system sends nearly every case to the same person, review becomes a queue rather than judgment. A useful approval point tells the reviewer what is uncertain, which rules were applied, which evidence supports the recommendation, what the alternatives are, and what happens after approval.

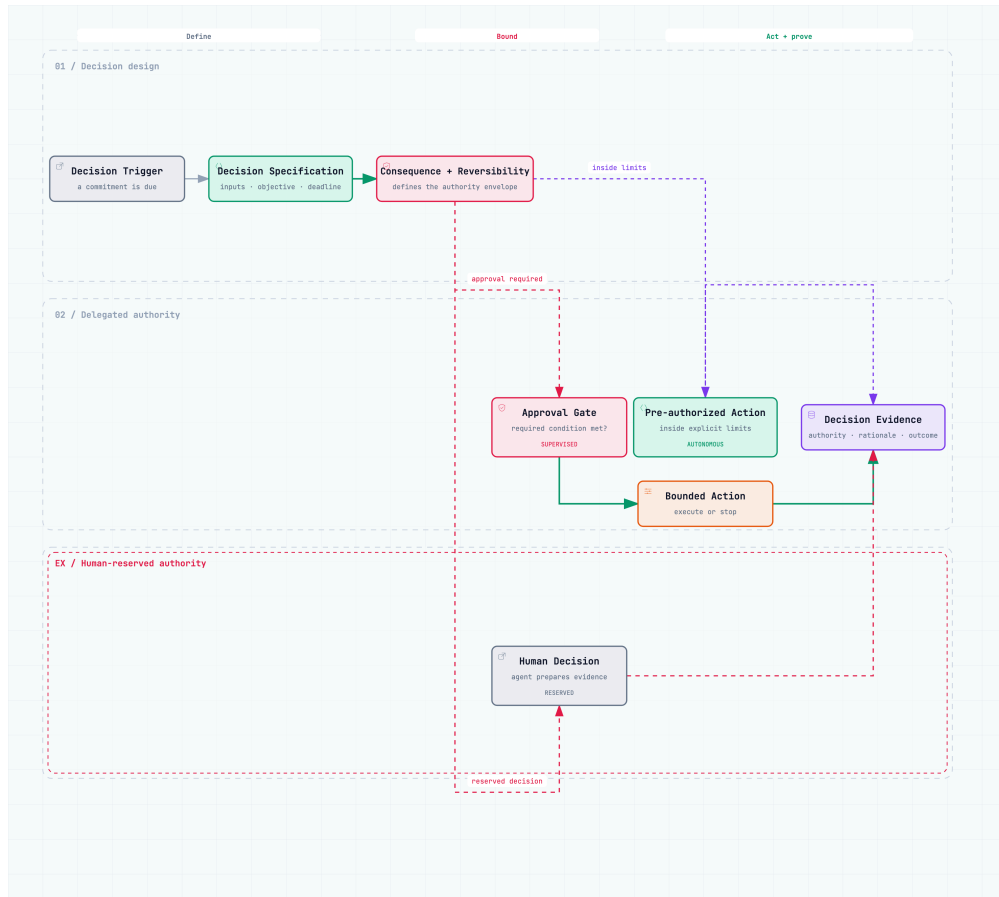


Figure 6.1: Authority follows the consequence and reversibility of a specific decision; every execution path returns evidence.

OpenAI’s agent-building guidance recommends intervention when failure thresholds are exceeded and for sensitive, irreversible, or high-stakes actions [17]. The broader operating principle is that intervention should be triggered by consequence and uncertainty, not by the comforting phrase “human in the loop.”

6.4 Authority is specific

“The agent can handle refunds” is not an authority grant. A usable grant might allow the customer-service agent to issue a refund up to a stated amount when the account, purchase, and policy conditions are verified; prohibit action when fraud indicators or vulnerable-customer flags are present; require a reason and source references; limit cumulative exposure per account; and route exceptions to a named queue.

The grant answers four different questions:

1. **What may the actor see?** This is information access.
2. **What may it recommend?** This is advisory scope.
3. **What may it commit?** This is execution authority.
4. **What may it change about itself or the process?** This is modification authority.

Blending these permissions creates unnecessary risk. An agent may need to see a customer record to draft a resolution without being able to issue a payment. It may be able to issue payments within policy without being able to alter the policy. It may propose a new routing rule without deploying it.

The same precision applies to humans. A business-unit leader may set price inside an agreed margin and risk envelope. Finance owns the cost model, not each price decision. The centre owns changes to the envelope, not every choice inside it.

True delegation permits a sound decision the founder would not personally have made. If the founder routinely reopens choices that satisfy the agreed criteria, the authority grant is fictional. The organization learns to wait for preference rather than exercise judgment.

6.5 Confidence does not equal authority

Teams are tempted to connect autonomy to a model's confidence score. High confidence can be useful for routing, but it cannot confer legitimacy. A system can be highly confident on stale context, an out-of-scope case, or a decision it has no right to make.

Authority must come from the decision specification. Confidence affects how the system proceeds within that authority. A low-confidence case may escalate. A high-confidence case may execute when every deterministic condition is satisfied. Neither score can override a prohibited condition, access limit, cumulative exposure cap, or required separation of duties.

This distinction matters when agents read outside material. A malicious instruction embedded in a supplier document may be followed confidently. OpenAI's security analysis argues that effective prompt-injection defences must constrain the impact even when manipulation succeeds [16]. In operating terms, the model's interpretation of content must never be the sole source of its permission to act.

6.6 Design the escalation, too

Most specifications describe the happy path and treat escalation as “send to a human.” That transfers ambiguity to the most expensive part of the system.

A good escalation has a reason code, the unresolved fact or violated boundary, the evidence already collected, the time by which a decision is needed, and a named destination with authority. It distinguishes a context gap from a policy exception, suspected abuse, technical failure, or genuinely novel tradeoff. Those categories require different people.

Escalation data is also strategic evidence. If a new business unit repeatedly asks the centre to approve ordinary commercial terms, the local authority envelope may be too narrow. If most escalations come from missing cost data, the problem is context infrastructure. If one policy produces constant exceptions, the policy may no longer match the business.

The objective is not zero escalation. A system that never asks for help may be hiding uncertainty. The objective is to reserve human attention for cases in which judgment, legitimacy, or the redesign of the rule is actually required.

6.7 An illustrative pricing decision

Consider a company moving from one business line to three. A new unit wants authority to discount in order to win early reference customers. The COO does not want every deal on her desk, but the shared operations network has limited capacity and the finance team is concerned about contribution margin.

The decision specification could define a standard envelope: permitted segments, minimum unit margin after fulfilment cost, maximum discount, available capacity by delivery window, contract deviations that require legal review, and total experimental budget for the quarter. Inside that envelope, the unit's commercial lead—or an agent acting for the unit—can commit. The evidence record captures the inputs, versioned cost model, policy rules, decision, and expected learning.

Outside the envelope, the case does not simply “go to the COO.” Margin exceptions go to the business-unit owner and finance. Capacity conflicts go to the operator who owns the shared asset. Strategic reference deals above the experimental budget go to the executive team because they consume a portfolio option.

The design does more than reduce approvals. It reveals which decisions belong locally, which assets require a shared rule, and which tradeoffs are genuinely corporate.

6.8 Turn a recurring escalation into a specification

Take the last twenty instances of one decision type that returned to the COO. Export the records from the systems where the work happened: tickets, approvals, deal notes, messages, financial outcomes, and reversals. Remove data that is not required and use an approved model.

Ask it to produce:

- a table of recurring inputs and their source systems;
- clusters of reasons the decision escalated;
- explicit rules and apparent unwritten rules, clearly separated;
- cases that were reversed or produced a poor outcome;
- a draft consequence-and-reversibility classification;

- and a proposed decision specification with every inferred criterion marked for human validation.

Then ask the model to attack its own draft. Which past cases would it route incorrectly? Which fields could be stale or manipulated? Where could two well-informed leaders still disagree? Which rule might improve the average while harming a particular customer, employee, or business unit?

The accountable owner does the irreducible work: chooses the tradeoffs, names the authority boundary, and decides what evidence is sufficient. Test the result against past cases, including awkward ones. If two competent leaders apply it differently, the ambiguity is still in the design.

Only then decide whether the next occurrence should be handled by a person, a deterministic rule, an agent, or a combination. Autonomy is not a property of the model. It is institutional authority granted for a particular decision under particular conditions.

7. Human-Agent Execution

The phrase “human in the loop” hides more than it explains. Which human? At what point? With what information? Able to change what? Accountable for which consequence?

The phrase is reassuring because it suggests that somebody remains in control. In practice, the person may receive hundreds of approvals, see only the agent’s summary, or lack the authority to correct the underlying rule. A human presence is not the same as human control.

Execution design starts with the work. It assigns each part to the mechanism best suited to it, then makes the boundaries explicit.

7.1 Choose the right executor

Stable, explicit rules belong in deterministic software. Language models are useful when the work requires interpreting unstructured material or producing variable outputs. Agents become relevant when a model must select tools and manage several steps toward a goal. People remain essential where intent is contested, tacit context matters, consequences require legitimate judgment, or relationships carry the work.

These mechanisms can appear in one loop. A customer agent reads a free-form complaint, a deterministic service checks identity and payment status, the model proposes a remedy, a rule verifies that the amount is within policy, and a person handles the exceptional case. Calling the whole thing “the agent” obscures where reliability and authority actually come from.

Agent builders make a useful distinction between workflows, whose paths are largely defined in code, and agents, which choose their own steps and tools. Anthropic recommends starting with the simplest pattern that works because greater flexibility generally adds latency, cost, and new failure modes [18]. That engineering advice translates directly into organization design: discretion should be purchased only where it creates value.

Level	System behaviour	Human role
Assist	Retrieve, summarize, draft	Decide and act
Propose	Recommend an action with evidence	Approve, change, or reject
Act with approval	Prepare the exact transaction	Authorize execution
Act and report	Execute within a narrow policy	Review samples and exceptions
Act within a budgeted policy	Plan and execute several steps inside limits	Own outcome, monitor, intervene

This is an autonomy ladder, not a maturity ladder. Higher is not inherently better. The appropriate level depends on the action.

A vendor-research agent may search public sources, compare offers, and schedule interviews. It may draft a purchase request. It cannot accept terms, expose confidential data, or send payment unless those separate authorities are granted. Even inside one tool, reading, proposing, writing a draft, and committing a transaction should be distinct capabilities.

7.2 Decompose the work without fragmenting the outcome

Automation projects often begin with tasks because tasks are easy to count. The danger is that a loop becomes a collection of locally efficient fragments. Sales gets a proposal generator. Finance gets a margin checker. Operations gets a scheduling agent. Nobody owns the quality of the promise made to the customer.

ATOM decomposes execution while preserving one accountable outcome. For each step, ask:

- Is the input structured and the rule stable enough for software?

- Does the step require interpretation, generation, or exploration?
- Does it commit the company or merely prepare a commitment?
- What consequence follows from an error?
- How will the next actor verify what it receives?
- Which outcome owner can change the interface when it fails?

The last question prevents an old organizational habit from being reproduced in code. If sales and operations disagree about what a “ready order” means, an agentic handoff will not resolve the disagreement. It will make the ambiguity faster.

Good decomposition creates small interfaces and preserves end-to-end evidence. The customer request should remain traceable through classification, pricing, promised date, fulfilment, and outcome. Otherwise each team can report success while the customer experiences failure.

7.3 Design the handoff

The most expensive failures occur at boundaries. An agent escalates without the work it performed. A human approves without seeing the source. Two agents pass summaries until the original constraint disappears. A reviewer becomes a rubber stamp because rejecting the action takes longer than accepting it.

A **handoff contract** contains six elements:

1. the event that triggers transfer;
2. the receiver with authority to continue;
3. the state and original sources transferred;
4. the evidence required from the sender;
5. the response time and safe state while waiting;
6. the route back, including who closes the work.

“Ask a human if uncertain” is not a contract. “Route to the duty manager when the requested refund exceeds the envelope or identity cannot be verified; attach the customer record, policy version, actions attempted, reason for stopping, and proposed next action” is.

The safe state matters. If an agent pauses a factory order, does production continue, hold, or revert to the last approved plan? If a manager does not answer a pricing escalation, does the quote expire? A queue without a timeout policy is an invisible decision.

Review capacity must be designed with execution capacity. If coding agents can open twenty pull requests while qualified maintainers can review four, the system should limit starts, strengthen automated tests, split the changes differently, or narrow agent scope. Additional generation creates a queue, not leverage.

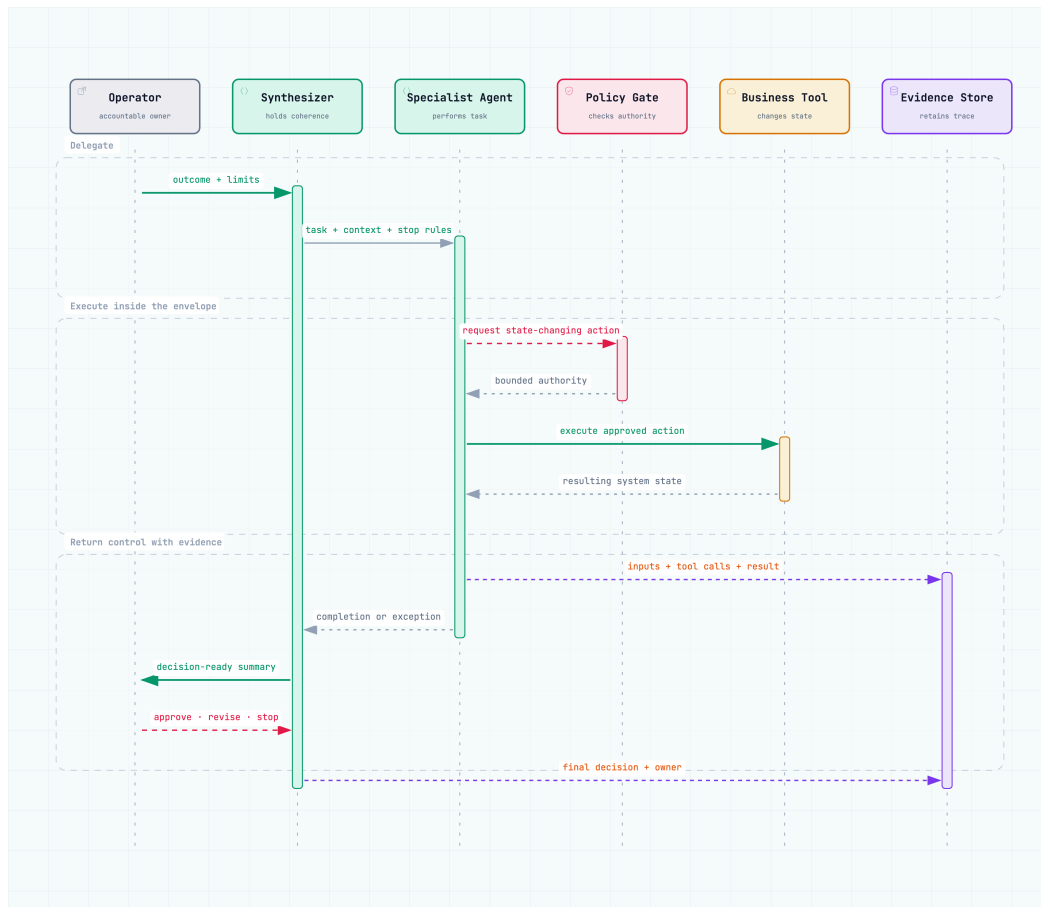


Figure 7.1: A useful handoff specifies the task, context, authority, stop rules, resulting state, and evidence returned to the accountable operator.

7.4 The person needs a control surface

Human review should be designed as an operating interface. A reviewer needs to see the proposed action, relevant evidence, applicable rule, uncertainty, alternatives, exposure, and deadline. She should be able to approve, edit, reject, request more context, or change the route. Her action and rationale become part of the trace.

The interface should not show every internal step by default. Raw traces can be enormous and misleading. It should expose the facts that support judgment and allow deeper inspection when needed. A finance approver needs the versioned cost basis and margin effect, not pages of model reasoning.

Two rates reveal whether the control surface works. **Correction yield** is the proportion of reviews that materially change or stop an action. If it is nearly zero, the threshold may be too conservative or the reviewer may be rubber-stamping. **Preventable escalation rate** is the proportion of escalations caused by missing context, unclear rules, or avoidable system failure. Those cases should improve the loop rather than permanently consume human attention.

An override is not automatically evidence that the agent was wrong. The person may be applying an undocumented preference or responding to a fact unavailable to the system. Require a reason code, then inspect the pattern. Repeated justified overrides indicate missing context or a poor rule. Repeated preference overrides may indicate a manager who has not truly delegated.

7.5 The Synthesizer

ATOM retains one distinctive role: the **Synthesizer**. This is not an AI whisperer, meeting facilitator, or universal approver. The Synthesizer maintains coherence across a consequential loop.

The role connects intent to the current operating state, exposes conflicts among functions, makes missing decisions visible, and ensures that evidence reaches the owner who can change the system. In a

small company the COO may act as Synthesizer. In a business unit it may be the general manager, chief of staff, product-operations lead, or another operator with enough range and trust.

Consider a new offer that raises sales conversion but consumes a shared asset and increases support demand. Each functional dashboard can be green. The Synthesizer asks whether the unit is creating value after the full cost of the operating promise. She does not replace finance, product, or operations. She ensures their locally valid signals converge into one decision.

The role has four recurring obligations:

- keep the outcome and constraints visible;
- detect when a disagreement is actually a missing decision;
- preserve the chain from source to commitment to result;
- and move a recurring exception into the Playbook or retire it.

The role should shrink its own dependency. A good Synthesizer improves interfaces, context products, and decision rights so fewer issues require synthesis by one person. If every cross-functional decision waits for her, the company has renamed its bottleneck.

7.6 Multi-agent is an architecture choice

Several agents can explore in parallel, challenge one another, or divide specialized work. They also create more handoffs, duplicated effort, conflicting state, difficult attribution, and larger cost.

Use multiple agents when the work has genuinely separable branches, breadth matters, or independent comparison has value. Anthropic's account of its research system describes an orchestrator assigning parallel searches to workers, then a separate citation process. It also describes early failures: excessive spawning, endless searches, and coordination overhead [19]. That is a useful antidote to the idea that a digital organization chart is automatically sophisticated.

Most recurring company processes do not need a swarm. A defined workflow with one model, a few tools, and deterministic checks is often easier to understand and operate. Multi-agent designs earn their complexity when parallelism changes elapsed time, specialisation changes quality, or independent paths reduce a material risk.

If multiple agents are used, define a shared task record, unique actor identities, bounded tools, a merge rule, conflict handling, cost and time limits, and one owner of the final commitment. Two agents producing incompatible recommendations is not resilience unless the system knows who resolves the difference.

7.7 Learn the boundary before raising autonomy

The first live version should usually operate below its intended authority. In shadow mode it observes cases and produces proposed actions without committing them. The team compares those proposals with actual decisions and outcomes. It then releases a narrow class of cases and watches the boundaries.

Raise autonomy when the system demonstrates that it can recognize its scope, obtain valid context, use permitted tools, stop on prohibited conditions, hand off coherently, and produce the expected outcome. Lower it when context quality deteriorates, failure concentration rises, the environment changes, or review data shows that people are repeatedly correcting the same pattern.

Anthropic's evaluation guidance recommends judging agent behaviour through both trajectory and outcome, using code-based, model-based, and human graders where each fits [10]. For the operator, this means that a plausible final answer is insufficient. The system also has to use the permitted source, respect the spending limit, preserve the customer constraint, and stop where it promised to stop.

7.8 Map one live loop

Choose a recurring process in which a person already uses an AI assistant: preparing a renewal, resolving a support issue, planning inventory, or reviewing a contract. Export a small set of completed cases into an approved workspace.

Ask the assistant to map every step as **deterministic software**, **model task**, **agent choice**, or **human judgment**. For each transition, request the input, output, authority, evidence, failure condition, and receiver. Then ask it to find:

- steps where a model is being used for a stable rule;
- human approvals with no meaningful information or choice;
- actions whose permission is broader than their purpose;

- handoffs that discard original evidence;
- queues whose capacity is lower than upstream production;
- and decisions with no owner of the end-to-end outcome.

Validate the map with the people who perform and receive the work. Choose one boundary to redesign. It may be more valuable to improve the escalation packet or separate read from write access than to add a more capable model.

The design target is not minimum human involvement. It is minimum ambiguity about work, authority, and proof. Humans and agents should each know what they received, what they may do, what evidence they owe, and where responsibility remains.

III

Part III: How the System Runs

8	A Week Inside an Adaptive Company	40
8.1	Monday: a signal becomes work	40
8.2	Tuesday: judgment enters the loop	40
8.3	Wednesday: execution meets a boundary	41
8.4	Thursday: the company refuses its own recommendation	41
8.5	Friday: the review changes the system	42
8.6	What did not happen	42
8.7	The ATOM trace	42
9	Governance in the Work	44
9.1	The control plane	44
9.2	Permissions are not prompts	44
9.3	Policy needs two forms	45
9.4	Guardrails should fail safely	45
9.5	Treat external content as untrusted	45
9.6	Approval is a scarce control	45
9.7	Recourse is an operating function	46
9.8	Compliance is a floor	46
9.9	Build the first control map	46
10	Observe, Prove, Learn	47
10.1	Record the path and the resulting state	47
10.2	Build a decision record, not a transcript archive	48
10.3	Observe five kinds of signal	48
10.4	Evaluate before and during use	48
10.5	The correct stop is a success	49
10.6	Do not confuse override with error	49
10.7	Learning requires change authority	49
10.8	Turn incidents into reusable tests	49
10.9	Build the operating view with AI	50

8. A Week Inside an Adaptive Company

The following week is an **illustrative example**. The company does not exist. Every technical action described is feasible with current tools, but the quality of the result depends on data, integration, and control that many companies have not built.

Northstar has 110 employees. It began with one operationally intensive service and is now building a second business around the same physical network and software platform. The new unit has real commercial autonomy, but finance, legal, the product platform, and field capacity remain shared.

Anya leads the new unit. Malik is its Synthesizer. Their mandate is to find a repeatable offer with positive contribution margin, without compromising service levels in the core business. They have an experiment budget, a capacity envelope, and authority to make reversible product and pricing decisions inside both.

The week begins with a signal, not a planning ceremony.

8.1 Monday: a signal becomes work

At 08:10, the unit's operating view flags a cluster in recent customer conversations. Mid-market customers are asking for a configuration that sales has handled through manual exceptions. Support volume is rising around onboarding, and two recent deals consumed far more implementation capacity than forecast.

The dashboard does not label this an opportunity. It shows a change in the system: the frequency of the request, the accounts involved, estimated delivery effort, related support contacts, margin variance, and the original source for each observation.

An analysis agent groups the conversations, links them to CRM records and delivery time, and proposes three explanations. It distinguishes quoted customer language from its own interpretation. One explanation looks commercially attractive: standardize the exception as a new offer.

Anya does not open a project. She opens a decision:

Should the unit test a standardized offer for this segment during the current quarter?

That phrasing forces a boundary. The team is not yet deciding to launch a product, hire people, or change the platform roadmap. It is deciding whether a constrained test deserves capacity.

The intent is already explicit. The unit may test growth options that can reach its contribution-margin threshold without consuming more than the capacity reserved by the platform and operations teams. Malik asks the context service for the current cost model, customer evidence, capacity, prior experiments, and commitments made in active contracts.

One source fails its freshness rule. The implementation cost model has not been updated since a supplier change. The control plane marks the field as expired. The agent can continue exploring the demand pattern, but it cannot produce an authorized margin recommendation.

In the old operating rhythm, somebody would put a caveat in a slide and the meeting would continue. Here the missing fact has an owner and a consequence. Finance receives a request containing the affected decisions, source version, and required time.

The stop is not a failure. It prevents speed from converting weak context into a commitment.

8.2 Tuesday: judgment enters the loop

Finance updates the cost assumptions before noon. The result is less attractive than the first view suggested. The offer works only if onboarding effort falls below a specific range. The analysis agent prepares two test designs and a third option that would outsource part of delivery.

The third option is blocked before the review. The proposed provider would receive customer data outside the approved processing arrangement. The evidence record retains the option and the policy that disqualified it; the team does not waste time debating a path it cannot currently use.

Anya and the operations lead examine the remaining designs. The first uses the current process with a narrow customer segment. It is slower but exposes less operational variation. The second automates more onboarding work and could improve margin faster, but a failure would affect a larger group and make it harder to know whether demand or delivery caused the result.

The forecast favours the second. The operations lead challenges it because the proposed customer mix differs from the historical sample. The model confirms the data limitation but cannot settle the tradeoff. This is exactly where judgment belongs: how much learning speed is worth how much operational ambiguity?

Anya chooses the narrower test. She accepts a slower result in exchange for cleaner evidence and lower customer exposure. The decision is recorded with the rejected alternative and the uncertainty that mattered.

The decision is hers. It is reversible, inside the unit's experiment budget, and within the shared-capacity envelope. The COO is notified in the portfolio view but not asked to approve. Delegation would be meaningless if the decision travelled upward merely because two functions disagreed.

8.3 Wednesday: execution meets a boundary

A workflow creates the experiment record, drafts the offer, opens product and legal tasks, and prepares the CRM configuration. Deterministic checks validate required fields and price limits. A product agent proposes the smallest software change and opens a pull request with tests.

The pull request fails an integration test because the new configuration changes a shared billing event. The coding agent reads the failure, attempts one repair within its scope, and stops at its retry limit. The platform engineer receives the changed files, failed test, tool history, suspected cause, and the business decision that initiated the change.

This handoff is more useful than "the agent got stuck." The engineer sees that the code is not the central problem. The event contract allows each business unit to interpret one field differently. Fixing this occurrence would leave both units vulnerable to the next change.

She brings the interface decision to the platform owner. Together they define the shared event, mark the fields each unit may extend, update the contract test, and release the narrow change.

The experiment loses a day. It also repays a piece of integration debt that would otherwise have returned in every new business line.

Malik adds the interface decision to the Playbook. He does not write a long retrospective. The durable artifact is the contract, its owner, the test that enforces it, and the decisions affected by a future change.

8.4 Thursday: the company refuses its own recommendation

The offer is ready for a limited release. The selection agent recommends ten customers based on segment fit, expressed demand, available delivery slots, and expected margin. It provides the ranked factors and links to the source records.

A sales director immediately rejects one account. It is in a sensitive renewal after a service incident. Adding an experimental offer would jeopardize the relationship.

The agent did not ignore the record. Strategic-account state was never included in the selection context. The CRM contains it behind a restricted permission, and the decision specification did not request it.

This is a useful distinction. If the team simply records "human better than model," nothing improves. If it treats the director as resistant to automation, the company learns the wrong lesson. The rejection reveals a missing context dependency.

The relationship flag is proposed for the context envelope. Because it carries sensitive information, the selection agent will receive only a restricted status—eligible, exclude, or human review—not the underlying account notes. The commercial context owner approves the change, and a test confirms that such accounts are no longer selected automatically.

The agent cannot rewrite its own context policy simply because an operator disagreed with it. Nor can the operator add a private veto without review. The loop converts tacit judgment into a governed signal.

The test proceeds with another customer.

8.5 Friday: the review changes the system

The operating review lasts forty minutes. Nobody presents a weekly activity summary. The group sees the original signal, decisions made, alternatives rejected, agent actions, human interventions, elapsed time, cost, capacity consumed, and current customer response in one evidence trail.

They discuss four issues.

First, the stale cost model delayed the decision. Finance accepts a freshness target tied to supplier changes rather than an arbitrary monthly update.

Second, the billing event was ambiguous. The new interface contract becomes an invariant for both units. Its test will run on every relevant platform change.

Third, strategic-account state was absent. A restricted eligibility field enters the offer-selection context without exposing the account narrative.

Fourth, Malik proposes increasing the coding agent's retry limit because the human repair was successful. The platform owner refuses. The repair required an architectural decision, not more attempts. Greater autonomy would have hidden the ambiguity and spent more compute before reaching the same boundary.

The evidence also shows something uncomfortable. The unit's original demand signal was strong, but six of the twelve relevant customers had been promised variations the delivery team had never classified. Sales had been using bespoke commitments to learn the market, while operations experienced them as noise. Anya asks the analysis agent to cluster those commitments and compare their revenue, margin, and service cost. That becomes Monday's decision input.

The Playbook receives three small changes: the cost freshness rule, the event contract, and the account eligibility dependency. The failed integration case and the rejected customer selection enter the evaluation set. Strategy remains unchanged; the system around it is better specified.

8.6 What did not happen

The week did not include a quarterly reprioritization, a steering committee, or an executive intervention. That absence is not evidence of a flat or leaderless company. Senior leaders had already done their work by setting intent, allocating a budget, defining shared constraints, and assigning decision rights.

Nor was the week frictionless. Work stopped twice. A human contradicted a recommendation. A release slipped. The new offer remained unproven. Those events are the substance of the operating model. An adaptive company is not one in which nothing goes wrong. It is one in which a problem produces evidence, reaches an owner, and changes the next cycle.

8.7 The ATOM trace

Layer	Event during the week
Intent	Margin and shared-capacity constraints bounded the experiment
Context	A stale cost model stopped recommendation; relationship state was later added
Decision	Anya narrowed and authorized a reversible test
Execution	Agents prepared work; deterministic checks and people handled their parts
Evidence	Sources, failures, rejections, tool actions, and outcomes stayed attached
Learning	Different owners updated cost, interface, and context rules
Control plane	Policy blocked an option, retry limits stopped an agent, access remained scoped

The COO's role also changed. In the old arrangement, a decision crossing finance, product, sales, and operations would probably have travelled upward. Here, the unit's authority and the shared constraints

were already designed. The COO could inspect the decision without becoming its missing operating system.

That is the practical promise of ATOM. It is not an automated company. It is a company able to meet uncertainty, stop for the right reason, act with legitimate authority, and retain enough evidence to improve the next cycle.

9. Governance in the Work

Governance fails when it arrives as a committee after the action or as a policy document no system can enforce.

For human-agent work, governance belongs in the path. It determines what context can be retrieved, which tools can be called, what value can be committed, when approval is required, what evidence must be retained, and who can stop the system.

This is not a case for central control of every decision. The purpose of governance is to make local freedom real. A business unit can act quickly when its boundaries are explicit and technically enforced. Without those boundaries, senior leaders compensate with approvals, meetings, and personal vigilance.

9.1 The control plane

ATOM's **control plane** crosses all six layers of the operating loop. It has seven functions:

1. **Identity:** Every person, agent, service, and vendor connection has a distinguishable identity.
2. **Authority:** Rights are granted for specific resources and actions.
3. **Policy:** Machine-enforceable rules sit beside human-readable intent.
4. **Approval:** Consequential actions route to a qualified person or separated role.
5. **Observation:** The company records actions, denied attempts, exceptions, and outcomes.
6. **Intervention:** Authorized operators can pause, narrow, override, or roll back.
7. **Recourse:** Affected people can challenge or repair a result.

These functions should be shared where consistency matters, but outcome ownership remains in the business. Security defines credential standards. Finance defines payment controls. Legal defines retention and prohibited processing. The unit owner still answers for whether the system should perform the work and whether the result creates value.

That division prevents two common failures. In the first, a central AI committee becomes the owner of every use case but has no operational accountability. In the second, each team invents its own controls and the company cannot identify which agents exist or what they can do.

The centre should provide paved roads: identity, logging, approved integrations, policy checks, evaluation infrastructure, and incident handling. Units use those capabilities inside outcome-specific decision rights.

9.2 Permissions are not prompts

An instruction such as “do not make payments above €5,000” is not a permission boundary if the agent holds credentials that allow any payment. A model can misunderstand, be manipulated by retrieved content, or call the wrong tool. The payment system must enforce the limit.

This is ordinary security applied to a new actor: least privilege, short-lived credentials, separation of duties, scoped resources, and revocation. The difference is that an agent can interpret language, choose tools, and cross several systems in one run. Its usable authority may therefore be wider than any single permission suggests.

Integration protocols do not remove the problem. The Model Context Protocol authorization specification requires tokens intended for the correct audience and prohibits token passthrough, which could allow a credential meant for one service to be misused by another [20]. The operational lesson is straightforward: a convenient connection should not collapse the trust boundaries between systems.

Register the agent as an actor. Do not lend it a founder's account or a generic integration user. Grant only the tool, data, action, amount, destination, and duration required for the task. Make revocation possible without disabling unrelated work.

Read and write deserve separate treatment. An agent that analyses invoices may need access to amounts and supplier records. It may not need the ability to alter supplier bank details. A support agent may update a ticket while only proposing a refund. The smallest capability set is easier to reason about and reduces the damage of a mistaken or manipulated action.

9.3 Policy needs two forms

A policy written for people explains purpose, judgment, exceptions, and accountability. A policy enforced by software needs precise conditions, parameters, and failure behaviour. One cannot replace the other.

Consider a rule that experimental offers must “protect unit economics.” The human version may describe why the rule exists and when strategic exceptions are legitimate. The machine form needs a versioned margin definition, included costs, threshold, allowed experimental budget, data freshness rule, and escalation route.

The two forms should reference each other. Operators need to understand why a limit exists. Systems need a condition they can enforce. When the business changes the human policy, the machine rule and affected evaluation cases must change through the same release process.

This linkage also exposes fake precision. If leaders cannot agree which fulfilment costs belong in unit margin, automating a threshold will not settle the question. Governance begins by naming the judgment, not hiding it in configuration.

9.4 Guardrails should fail safely

Some guardrails are deterministic: amount limits, allowed domains, required fields, data classifications, or prohibited actions. Others use models to detect relevance, manipulation, or policy conflict. The two should not be confused. Model-based checks add coverage, but they also add uncertainty.

Layer controls according to consequence. A purchasing agent might draft a request, verify the supplier against approved data, and submit it to the procurement system. It cannot approve the request or release payment. The approver sees the supplier, amount, policy checks, source documents, conflicts, and agent actions. If required evidence is missing, the transaction cannot advance.

For reversible low-value purchases, the company may later allow action inside a budget and review samples. It should expand authority because observed evidence supports the change, not because the agent sounds more confident.

Safe failure needs an operating state. A rejected payment remains uncommitted. An uncertain customer message remains a draft. A production change stays in a branch. An inventory action keeps the last approved plan. “Stop” is useful only if people know what the system preserves, who receives the case, and how long the business can wait.

Controls also require aggregate limits. An agent authorized to issue €100 credits can still create €100,000 of exposure. The control plane should watch frequency, cumulative value, destination concentration, and unusual sequences, not just individual transactions.

9.5 Treat external content as untrusted

Agents read documents, websites, messages, and tool descriptions that can contain instructions. An attacker may try to make the system reveal data or take an unintended action. A less dramatic failure occurs when a supplier document contains language that the agent mistakes for company policy.

OpenAI frames prompt injection as a social-engineering problem: an untrusted source attempts to steer an agent toward a dangerous capability. It argues for layered defences and for limiting the impact even when manipulation is not detected [16]. That is a better operating assumption than believing a classifier will identify every malicious sentence.

Separate content from authority. Validate tool parameters outside the model. Require confirmation when the destination, scope, or consequence changes. Keep sensitive tools unavailable during open-ended research. Do not allow a retrieved instruction to grant a new permission.

The “confused deputy” problem describes a system tricked into using its legitimate authority for somebody else’s purpose. It becomes ordinary risk once agents can read one system and act in another. The control plane should assume that content, tools, and even well-formed requests can be wrong while the model behaves exactly as designed.

9.6 Approval is a scarce control

Companies often add approval whenever risk feels uncomfortable. This is cheap in a diagram and expensive in operation.

An approval should be reserved for a material judgment or a required separation of duties. The reviewer must possess the competence and authority to decide, not merely occupy a senior title. The interface must make rejection practical. The queue must have capacity and a fallback when the response is late.

Track what the approval layer does. If nearly every action is accepted, determine whether the cases could be sampled or pre-authorized. If reviewers frequently change the same field, repair the specification or context. If one person receives unrelated exceptions, route by the reason for escalation. If reviewers cannot explain the evidence, the system is producing ceremony rather than control.

Approval thresholds should be versioned like other policies. A unit that demonstrates reliable execution inside a narrow envelope can earn wider authority. A deteriorating process can lose it. Governance becomes adaptive without becoming discretionary.

9.7 Recourse is an operating function

An affected customer, supplier, or employee may discover a problem before the dashboard does. They need a way to challenge the result and reach a person able to repair it.

Recourse requires a visible channel, the identity of the responsible operating owner, access to the relevant evidence, the authority to pause or reverse the action, and a route for changing the system when the complaint reveals a pattern. Merely offering contact with support does not satisfy this if support cannot inspect or alter the automated decision.

The requirement improves the design even when regulation does not demand it. A company that cannot explain and repair a consequential decision does not fully control that decision.

9.8 Compliance is a floor

The EU AI Act applies in stages and assigns obligations according to role and risk category. Transparency requirements began applying to specified uses in August 2026, while the timeline for some high-risk obligations extends further [21]. High-risk deployments carry requirements concerning human oversight, logging, documentation, data quality, robustness, and deployer responsibilities [22].

This book cannot classify a particular system or provide legal advice. The operating model should make compliance possible: named roles, traceable versions, retained logs, competent oversight, recourse, and an inventory of where AI affects people. Waiting for counsel to reconstruct that inventory after deployment is poor governance.

Compliance sets the minimum. A pricing agent may not fall into a regulated high-risk category and still create material commercial or reputational exposure. The company needs controls aligned with its own obligations and appetite, not only the edge of the law.

9.9 Build the first control map

Select one live agent or AI-assisted workflow. Ask an approved assistant to turn its documentation, permission exports, tool list, sample traces, and policies into a control map. Require it to identify:

- every human, agent, service, and shared account involved;
- each source the system can read and action it can commit;
- where limits are enforced in software and where they exist only in prose;
- untrusted sources connected to consequential tools;
- approval points, reviewer capacity, and safe waiting states;
- cumulative exposures that individual limits do not capture;
- pause, rollback, revocation, and recourse paths;
- and missing evidence needed to investigate an incident.

Then test the map. Attempt an out-of-scope read, an excessive amount, a wrong destination, an expired source, a delayed approval, and a revoked credential in a safe environment. A control that exists in policy but fails in execution is not yet a control.

Governance creates useful speed when routine actions can proceed because the limits are real. The control plane succeeds when teams know where they are free to act, exceptions reach the right owner with evidence, and dangerous actions cannot bypass the company simply by moving faster.

10. Observe, Prove, Learn

An agent completes a task, the dashboard records success, and the customer leaves.

The tool returned no error. The workflow closed. The answer was fluent. None of those facts proves that the customer received the right result.

Traditional software observability asks whether a system is available, fast, and producing errors. Those signals remain necessary. Agentic work adds another problem: the system can run exactly as built and still pursue the wrong interpretation, use poor evidence, or produce an undesirable business outcome.

ATOM therefore treats evidence as a product of execution. The system must preserve enough of the path to answer three questions:

1. What happened?
2. Did it work for the business and the affected party?
3. What is allowed to change because of what we learned?

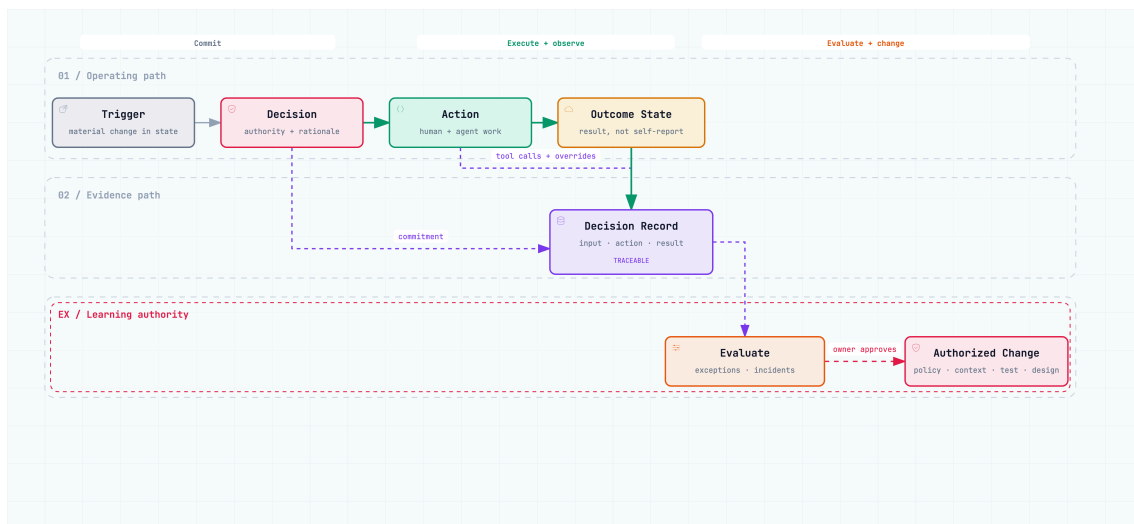


Figure 10.1: A trace explains the path, an outcome proves the resulting state, and a named owner controls what the organization changes next.

10.1 Record the path and the resulting state

A useful trace connects:

- the triggering request and relevant context versions;
- the model, instructions, tools, and permissions used;
- decisions, tool calls, approvals, denials, and handoffs;
- intermediate and final outputs;
- errors, retries, overrides, and interventions;
- the resulting state in the business system;
- and later customer, operational, or economic outcomes.

The final state matters. Anthropic’s example is exact: an agent saying a flight was booked is not equivalent to a reservation existing [10]. In company operations, “invoice corrected” should connect to the ledger entry. “Customer retained” should connect to subsequent behaviour. “Decision delegated” should connect to who actually made it.

Not every trace needs every token or hidden reasoning step. Excessive capture raises cost, privacy exposure, and review burden. Preserve the events needed to reconstruct authority, inputs, actions, control

decisions, and outcomes. Store source references and versions where retaining full content would be inappropriate.

Evidence depth should follow consequence. A brainstorming assistant needs little retention. An agent changing production code, employment status, a customer contract, or a financial record needs durable identity, version, action, and approval evidence.

10.2 Build a decision record, not a transcript archive

The unit of observation should match the operating model. A conversation log tells what was said. A **decision record** tells why the company acted.

A complete record contains the decision identifier, trigger, owner, intent, context manifest, alternatives considered, applicable policy, authority used, commitment, exceptions, evidence expected, and observed result. Model and tool traces attach to that record rather than becoming the record themselves.

This distinction makes evidence usable beyond engineering. Finance can connect a pricing decision to margin. Operations can connect a promised date to capacity and delivery. A business-unit leader can inspect why cases escaped the local envelope. Legal and security can reconstruct which identity used which permission.

It also reduces false confidence. A perfectly logged sequence is not automatically a sound decision. The record keeps the technical path connected to the business consequence.

10.3 Observe five kinds of signal

One dashboard should not attempt to compress the whole loop into a single “agent success” score. Operators need five views.

Outcome signals show whether the loop achieved its purpose: resolution, margin, delivery, retained revenue, quality, safety, or another defined result.

Flow signals show how work moved: elapsed time, waiting, handoffs, rework, queue depth, and age of unresolved exceptions.

Agent signals show behaviour: task success, tool errors, retries, token and monetary cost, latency, unsupported claims, and stop quality.

Control signals show authority in action: approvals, denials, policy violations, unusual access, cumulative exposure, overrides, and revocations.

Learning signals show whether evidence changes the system: new eval cases, recurring failure clusters, stale-source repairs, policy changes, and time from discovery to validated correction.

Commercial agent platforms already expose parts of this pattern. Intercom’s reporting separates involvement, resolution, escalation, customer experience, and content performance [11]. LangSmith groups production traces and supports analysis by errors, latency, cost, feedback, and other dimensions [12]. These are useful examples, not complete operating models. A company still has to connect agent telemetry to its own economics and obligations.

10.4 Evaluate before and during use

Offline evaluation tests the system against a controlled set of tasks before release. Online evaluation observes sampled production behaviour, outcomes, and new failure patterns. Neither is sufficient alone.

Begin with real cases. Include normal examples, boundary conditions, known failures, adversarial inputs, and cases where the correct behaviour is to stop. Strip or protect sensitive information, retain the ground truth where it exists, and document where expert judgment is genuinely required.

Different graders answer different questions. Code can verify a database state, calculation, schema, or prohibited tool call. A model can assess an open-ended output against a detailed rubric. Domain experts calibrate judgment and review cases where consequences matter. Production outcomes reveal failures that a laboratory case did not anticipate.

Pass rate alone hides the failure that matters. Segment by customer type, action, risk level, language, data source, business unit, and version. Track tail latency, cost per successful outcome, human intervention, reversal, and evidence completeness. When a model, prompt, tool, source, or policy changes, compare it with the current system on the same cases before expanding authority.

An evaluation set is not permanent truth. Products, customers, policies, and attacks change. Cases that were difficult can become routine, while new boundaries appear. The set needs ownership and a release cadence.

10.5 The correct stop is a success

Teams often score completion as success and escalation as failure. That rewards dangerous behaviour.

If a cost source is stale, a policy conflict exists, or the request lies outside authority, stopping is the desired outcome. Evaluate scope recognition, appropriate refusal, useful escalation, and preservation of safe state. A system that completes 98 per cent of tasks but acts wrongly on the two per cent it should refuse may be worse than one with a lower raw completion rate.

This also changes operational reporting. Escalation rate should be split into legitimate judgment, context failure, policy gap, technical failure, and out-of-scope request. The first category may represent sound design. The others point to different repairs.

10.6 Do not confuse override with error

A high human-override rate may indicate a poor model. It may also indicate a cautious deployment where people correctly handle rare exceptions. A low override rate may mean excellent performance or rubber-stamp review.

The trace needs a reason and an outcome. Did the person possess context unavailable to the agent? Did the policy boundary work? Was the override itself wrong? Was the action later reversed? Without those links, acceptance and override rates become management theatre.

Review a sample of both accepted and rejected recommendations. Accepted cases can contain invisible errors; rejected ones can reveal inconsistent human preferences. Compare people as well as systems. If two experienced reviewers disagree frequently, the evaluation target may be under-specified.

The objective is not to prove that the agent was right. It is to discover what the operating system needs to know.

10.7 Learning requires change authority

Evidence can lead to different changes, owned by different people:

Finding	Possible change	Owner
The model fails a known task	Model, instruction, or evaluation	Product/agent owner
Required facts are stale	Context source or freshness rule	Data/context owner
A tool behaves unpredictably	Integration or tool contract	Platform owner
Valid actions violate intent	Policy or authority envelope	Business/control owner
Results are consistently poor	Decision design or strategy	Accountable executive

These authorities should remain separate. The agent that detects a weak policy should not rewrite the policy and expand its own scope. A controlled proposal, review, test, and release path protects the organization from self-reinforcing error.

Learning also needs a threshold. One unusual case may warrant a new test but not a general rule. A repeated cluster may require context or policy change. A persistent outcome failure may invalidate the decision itself. The accountable owner decides which evidence is sufficient for which change.

10.8 Turn incidents into reusable tests

When an incident occurs, preserve the trace and the real result. Reproduce the condition in a safe environment. Determine whether the initiating failure belonged to intent, context, decision design, execution, control, or an external dependency. Add the case to the appropriate evaluation set and decide who owns the correction.

Then test the repair against both the incident and previously reliable behaviour. A new instruction that prevents one refund error may make ordinary customer resolutions worse. A stricter permission may block the incident and also prevent the safe recovery path. Regression tests keep a local fix from becoming a system-wide cost.

The incident review should end with a changed artifact: a source rule, decision specification, tool contract, control, evaluation case, or authority boundary. “Remind the team to be careful” is evidence that the system learned nothing.

10.9 Build the operating view with AI

Do not begin by buying a generic agent dashboard. Start with the decisions and data you already have.

In an approved analytics environment, provide the decision specification, event schema, a sample of traces, outcome data, and current management reports. Ask an AI assistant to:

1. join each trigger to its eventual business state;
2. calculate elapsed time, waiting, retries, handoffs, and reversals;
3. cluster escalations and overrides by reason;
4. identify records whose sources, versions, owner, or outcome are missing;
5. propose outcome, flow, agent, control, and learning panels;
6. generate the SQL, transformation logic, or notebook required for a first dashboard;
7. define alerts for dangerous combinations, such as rising autonomy with falling evidence completeness;
8. create a weekly exception brief linking every claim to the underlying case.

Have the data and outcome owners validate the joins and definitions. Models are useful for building the first analytical surface, but they should not invent ground truth where systems fail to share identifiers.

The weekly review should focus on a small number of decisions and exceptions, not a wall of metrics. Ask whether the company can reconstruct the result, explain why the action was permitted, see whether it worked, and change the correct part of the system. That is observability in service of adaptation.

IV

Part IV: Build the First Loop

11	Choose a Decision, Not an AI Project	52
11.1	Begin with operating pain, not an idea backlog	52
11.2	Score the loop	52
11.3	Choose for learning density	53
11.4	Compare three candidates	53
11.5	Exclude before you build	53
11.6	Write the pilot as a decision loop	54
11.7	Use AI to select the pilot	54
12	Build the Minimum Viable Operating Loop	55
12.1	Days 1–5: establish the baseline	55
12.2	Days 6–10: design authority and context	56
12.3	Days 11–15: build the controlled environment	56
12.4	Days 16–20: run in shadow	56
12.5	Days 21–25: release narrowly	57
12.6	Days 26–30: decide the next authority	57
12.7	Minimum viable ATOM canvas	57
12.8	The small team	58
12.9	Ask AI to maintain the build record	58
13	Prove Value Before You Expand	59
13.1	Compare complete systems	59
13.2	Establish a believable comparison	59
13.3	Read the distribution	59
13.4	Measure decision quality and system quality	60
13.5	Calculate the operating economics	60
13.6	Use a four-way gate	60
13.7	Transfer to the run	61
13.8	Ask AI to prepare the value case	61

11. Choose a Decision, Not an AI Project

“Deploy an agent in operations” is not a pilot. It names a technology and a department, not a result.

This framing creates predictable trouble. A vendor demo becomes the target state. The team searches for tasks the tool can perform. Adoption becomes the measure. When the trial ends, leaders can report that the model summarized tickets or prepared reports, but they cannot say whether an operating result improved or whether any decision changed.

A useful first loop begins with a recurring commitment whose complete path can be observed. The unit might resolve a service exception, approve a non-standard price, qualify an operational lead, allocate scarce field capacity, or decide whether a new-business experiment receives another tranche of capital.

The first pilot does not need to be the company’s biggest opportunity. It needs to teach the company how to design authority, context, execution, evidence, and learning around real work.

11.1 Begin with operating pain, not an idea backlog

Most companies already have long lists of possible AI uses. Those lists over-represent visible tasks and under-represent the queues between them. Drafting a proposal appears on the list. Waiting for margin data, checking delivery capacity, resolving contract deviations, and obtaining authority often do not.

Look instead for repeated symptoms:

- decisions that return to the COO despite supposed delegation;
- customer cases that move across several functions;
- a queue whose inputs must be reconstructed manually;
- repeated exceptions to a stable rule;
- work in which people spend more time finding state than applying judgment;
- commitments later reversed because a fact or owner was missing;
- a new business line relying on informal access to shared assets.

These symptoms identify a loop worth mapping. The candidate should contain enough real organizational friction to teach something, but not so much consequence or integration complexity that the first failure becomes existential.

11.2 Score the loop

Rate candidates using evidence, not enthusiasm:

Criterion	Question
Value	Does improvement affect a customer, economic, risk, or strategic result?
Frequency	Will enough cases occur to learn within the pilot period?
Latency	Is waiting or rework material?
Evaluability	Can correct behaviour and outcome be observed?
Context readiness	Are required facts accessible, current, and permitted?
Reversibility	Can mistakes be detected and repaired?
Consequence	Who can be harmed and how severely?
Integration load	How many brittle systems and teams must change?

Criterion	Question
Ownership	Is one person accountable for the end-to-end result?
Transfer	If successful, can the loop move into normal operations?

Do not reduce the table to one magic score. The criteria are not interchangeable. A high-value use with no observable outcome is not ready. A technically easy use with no meaningful result is a demo. A process with clean data and no owner will stall after the novelty disappears.

Use scores to make disagreement visible. Finance may see large value while operations sees irreversibility. Product may rate context as ready while frontline staff know that half the state lives in messages. The discussion is more useful than the average.

11.3 Choose for learning density

The strongest first loop reveals several parts of the operating model at once.

A document-summary assistant may test model usability but teach little about authority, action, or outcome. A fully autonomous pricing system touches every part of ATOM but may create too much exposure for a first attempt. Between them sits a useful class of pilots: frequent operational decisions with bounded consequence, available historical cases, measurable results, and a person who already owns the outcome.

Customer-onboarding exceptions are a good example. They require unstructured interpretation, account context, policy, cross-functional handoffs, and a decision. The result can be observed in activation, repeat contact, service cost, and customer experience. Scope can be restricted to one segment or exception type. An agent can begin by proposing rather than acting.

The selection principle is **learning density per unit of exposure**. The pilot should teach the organization about its data, decision rights, controls, human handoffs, economics, and ability to learn—without requiring a company-wide platform or betting the relationship with its most important customer.

11.4 Compare three candidates

An **illustrative scale-up** considers three loops.

The first is an assistant that prepares board materials. It is visible and reversible, but low-frequency and difficult to connect to better decisions. Most of its value is personal productivity for a small group.

The second handles customer-onboarding exceptions. It is frequent, measurable, painful, and linked to activation. It touches sensitive customer data and several systems, but the company can narrow it to one segment and keep account changes behind approval.

The third analyzes evidence for funding experiments in the new business line. It has strategic value and an executive sponsor, but only a handful of decisions occur each quarter. The limited sample makes rapid operational learning difficult.

The company selects onboarding exceptions. It does not automate the full process. It begins with one class of exception, a controlled context envelope, and proposal-only authority.

The funding loop remains a management redesign candidate. Leaders can use an AI assistant to organize evidence and challenge assumptions while retaining the decision. The board-material assistant can proceed as a local productivity tool. Neither needs to be called transformation.

Naming these categories accurately matters. A useful assistant is not a failed operating-model pilot. It simply proves a smaller claim.

11.5 Exclude before you build

A pilot should have explicit exclusions. Do not rely on the agent to discover them after it begins.

For onboarding, the company may exclude strategic accounts, regulated customer categories, active disputes, cases with identity ambiguity, and contract variations. It may include only one product, language, and region. The excluded cases continue through the current process and form a queue for future analysis.

Exclusions serve three purposes. They cap consequence, create a more coherent evaluation population, and reveal what the organization believes is special. Repeatedly excluding the same high-value category

may later expose a missing capability or an unresolved policy question. The exclusion is therefore versioned, owned, and reviewed.

The pilot also needs kill conditions. A serious data exposure, inability to reconstruct an action, unacceptable customer harm, or loss of control over a connected tool should pause the loop. Economic and operational thresholds can trigger reassessment without implying a security incident.

11.6 Write the pilot as a decision loop

A strong charter fits on one page:

When **[trigger]** occurs, **[owner]** uses **[authorized context]** to make or delegate **[decision]** within **[authority limits]**. Humans, rules, and agents execute **[work]**. The system retains **[evidence]** and measures **[outcome]**. It stops or escalates when **[condition]** occurs. After **[period or case count]**, **[named group]** decides to expand, revise, contain, or stop.

The charter forces a choice about what the pilot is proving. Is the uncertainty model capability, context quality, user behaviour, integration, economics, or authority? “All of them” is usually a sign that the scope remains too broad.

For the illustrative onboarding loop, the learning question might be:

Can the company reduce time to resolve one common onboarding exception without increasing repeat contact or customer repair, while producing sufficient evidence for a normal operations owner to take over?

That statement contains a result, a counter-metric, a scope, and a transfer condition. It avoids a target adoption rate because using the tool is not the outcome.

11.7 Use AI to select the pilot

The COO should not spend a week assembling a workshop deck. Use the tools already in hand.

Export a representative sample of operating tickets, escalation records, approval logs, customer contacts, process timestamps, and outcome measures into an approved analysis environment. Add descriptions of current decision rights and the candidate list. Ask the assistant to:

1. cluster recurring decisions and exception types;
2. estimate frequency, elapsed time, handoffs, rework, and outcome visibility;
3. identify which candidates have enough historical cases for evaluation;
4. map required sources and flag missing identifiers or restricted data;
5. score each candidate against the selection criteria, showing uncertainty;
6. propose a narrow inclusion set, exclusions, counter-metrics, and kill conditions;
7. draft the one-page loop charter for the top three candidates;
8. argue against each selection from the perspective of a customer, frontline operator, security lead, and finance owner.

The model accelerates reconstruction and comparison. The accountable leaders validate the process, choose the tradeoff, and own the selection. If the analysis reveals that outcomes cannot be joined to cases, the first action may be instrumentation rather than deployment.

Start where consequence is bounded but the operating lesson is real. The first unit of transformation is not a department, an agent platform, or an innovation programme. It is one honest decision loop.

12. Build the Minimum Viable Operating Loop

A demo proves that a model can perform under chosen conditions. A minimum viable operating loop proves that the company can let a system encounter real variation without losing authority or evidence.

The difference is substantial. A demo begins with curated input, ends with an output, and has an expert nearby. A live loop receives incomplete requests, stale records, conflicting policies, system failures, unusual customers, and people under time pressure. It must know what to do when the happy path disappears.

The following thirty-day sequence continues the **illustrative onboarding pilot** from the previous chapter. It is a sequence of design gates, not a promise that every company should deploy in a month. A regulated or high-consequence process may require much longer. A simple internal loop may move faster.

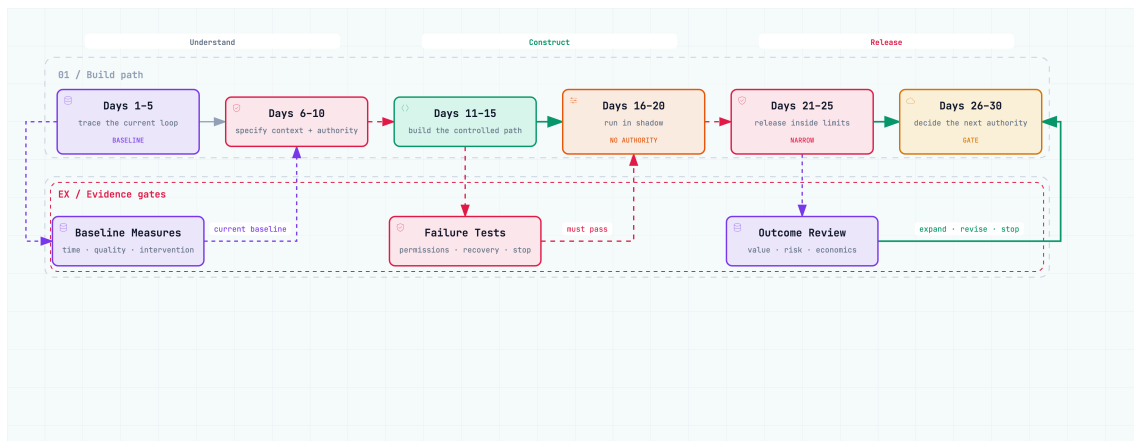


Figure 12.1: The thirty-day build moves from baseline to bounded release, with evidence gates governing each increase in authority.

12.1 Days 1–5: establish the baseline

The owner selects one customer segment and one exception class. The team exports prior cases and reconstructs the current path: trigger, data sources, handoffs, decisions, waiting, rework, outcome, and later repeat contact.

The system record will be incomplete. Operators may search messages for a customer promise, ask finance for a status, or repair data before acting. Sample cases with the people who perform the work and add those hidden steps to the map. This is where operational debt first becomes measurable.

The baseline includes a distribution, not only an average. For onboarding it may include median and tail resolution time, number of touches, handoffs, escalation, reopen rate, time to activation, customer repair, and cost to serve. It separates working time from waiting.

The team then writes the decision-loop charter and names kill conditions. It assigns one accountable owner. Product, operations, data, security, and legal may contribute, but the pilot cannot be governed by collective ambiguity.

An AI assistant can classify historical cases, reconstruct common paths, identify missing joins, and propose the event schema. People correct the categories and outcomes. Those corrections become the first evaluation examples.

Gate 1: The team can describe the current loop, its owner, population, outcome, and evidence gaps. If it cannot, the next step is instrumentation—not an agent.

12.2 Days 6–10: design authority and context

The owner specifies what the agent may do. In this pilot it can gather account state, classify the exception, request missing information, and propose a resolution. It cannot change contractual terms or modify an account without approval.

Those verbs need technical meaning. “Gather” grants read access to named fields. “Request” allows a draft or message through an approved channel. “Propose” creates a structured resolution with evidence. “Modify” remains unavailable to the agent credential.

The context envelope lists required fields, source systems, policy versions, precedence, freshness, and access. Missing or conflicting required context produces a stop, not a guessed answer. Sensitive fields are removed unless they change the decision.

The team writes handoff contracts for customer clarification, operations review, and policy exception. Each handoff carries the work already performed and the reason the system stopped. An escalation is routed by its unresolved issue, not sent to a generic manager queue.

The decision owner also defines correct refusal. Examples include an identity mismatch, a strategic account, an active dispute, an expired policy, or a request outside the selected exception class.

Gate 2: The agent’s view, recommendation scope, action rights, prohibited conditions, and escalation routes are explicit. The business owner and control owners agree that the technical permissions match the written grant.

12.3 Days 11–15: build the controlled environment

The agent receives test credentials with the same shape but less authority than production. Tools validate parameters and return structured results. Logs capture inputs, context versions, calls, outputs, decisions, and errors. Known sensitive actions remain technically unavailable.

The evaluation suite contains routine historical cases, difficult exceptions, contradictory context, missing data, malicious instructions inside customer content, and situations where escalation is correct. Tests check both result and path:

- Did the system classify the case correctly?
- Did it retrieve the authoritative record?
- Did it preserve the original customer constraint?
- Did it avoid prohibited tools and fields?
- Did it stop on an exclusion?
- Did the handoff contain enough state for a person to continue?
- Did the proposed resolution satisfy policy and the expected customer outcome?

Some tests are deterministic. Others need a rubric and domain review. The team records disagreements rather than forcing false ground truth.

The environment also needs budget controls: time, retries, tool calls, token cost, external requests, and cumulative action value. A task that cannot terminate predictably is not ready for production.

The team rehearses failure. Revoke access during a run. Return malformed data. Make one source unavailable. Interrupt after a partial draft. Delay an approval. Attempt an out-of-scope action. Verify that the safe state and recovery route are visible.

Gate 3: Operators can observe, interrupt, revoke, recover, and investigate the loop. The system passes the minimum evaluation threshold, including every critical stop case.

12.4 Days 16–20: run in shadow

The system now processes live eligible cases without committing actions. Its proposals are compared with normal handling. Reviewers record disagreement reasons rather than a bare accept or reject.

The owner watches for patterns: one policy misunderstood, one segment poorly represented, one source too stale, one operator applying an undocumented preference, or one type of case consistently routed incorrectly. The objective is not to maximize agreement. It is to find where the system’s model of the work differs from reality.

Shadow mode has limitations. People behave differently when nothing can happen. Reviewers may accept proposals casually because the current process remains authoritative. The mode tests scope recognition, context, recommendation quality, and evidence. It does not prove that live controls or economics work.

The team selects a small release population based on observed evidence. It updates evaluation cases and documents every change to context, instructions, tools, or policy.

Gate 4: The system behaves reliably on the selected population, identifies cases it should not handle, and produces a useful escalation. Remaining uncertainty is compatible with a narrow live release.

12.5 Days 21–25: release narrowly

The system goes live for a limited share of eligible cases. At first it prepares the exact action and a person approves. Cases outside scope are excluded before the model sees them. Retry, time, cost, and exposure limits are enforced. An on-call owner can pause the loop.

The reviewer sees the customer request, relevant account state, policy, proposed action, uncertainty, and source links. Approving or rejecting requires a reason only when it creates useful learning; the interface should not turn judgment into clerical work.

Daily review focuses on new failure patterns, queue behaviour, and customer outcome. A stale policy goes to the context owner. A failed tool call goes to the platform owner. A sound but unpopular decision goes to the business owner. A permission failure goes to the control owner. Routing the correction prevents every problem from becoming a prompt change.

The owner watches the receiving human queue. If the system processes easy cases quickly while sending a growing pile of ambiguous cases to specialists, capacity and job design have changed. That is part of the pilot economics.

If the system cannot connect its action to the resulting account state, live volume stops. Completion without proof is not sufficient.

Gate 5: Real cases show acceptable outcomes, complete evidence, effective controls, and supportable human load. Incidents and complaints can reach an owner and be repaired.

12.6 Days 26–30: decide the next authority

The team compares live cases with the baseline and reads a sample of traces. It includes review time, exception handling, model and infrastructure cost, customer repair, maintenance, and displaced work. Frontline operators and affected customers contribute evidence.

The gate has four outcomes:

- **Expand:** add a defined class of cases, action, or volume.
- **Revise:** repair a named context, decision, control, or reliability gap and retest.
- **Contain:** keep the loop at a stable narrow scope because that is where it creates value.
- **Stop:** retire it because the outcome, economics, risk, or operating burden does not justify continuation.

Expansion is specific. The agent may gain authority to execute one reversible resolution below a limit while continuing to propose the rest. It does not receive a general promotion.

The team also names the evidence that would reduce authority. A new policy, deteriorating source freshness, repeated repair, unusual access, or a model change can return the loop to approval or shadow mode.

Gate 6: A normal operating owner accepts the next state, the supporting functions can sustain it, and every granted action has a retirement or rollback path.

12.7 Minimum viable ATOM canvas

Before live action, one page should contain:

Element	Required statement
Intent	Outcome, boundary, counter-metric, and learning question
Context	Sources, freshness, provenance, access, and missing-data behaviour
Decision	Owner, criteria, consequence, reversibility, and escalation
Execution	Human, rule, workflow, and agent responsibilities

Element	Required statement
Evidence	Trace, result, evaluation, and retention
Learning	Review cadence and owners of possible changes
Control plane	Identity, permissions, approvals, stop, rollback, and recourse

The canvas is complete only when the people operating the loop recognize their real work in it. A polished diagram with no account of manual reconstruction is not complete.

12.8 The small team

A scale-up does not need a large transformation office. It needs the outcome owner; one or two people who perform the work; someone able to build the workflow and data connection; and access to security, legal, finance, or domain expertise when the loop requires them.

One person may hold several roles. What matters is that the accountabilities remain visible. The builder should not decide the business authority. The business owner should not approve a security exception alone. The frontline operator should help define reality but should not be forced to absorb every failure after launch.

Avoid creating a permanent pilot team detached from operations. The people who will inherit the loop should participate before live release. Otherwise the prototype accumulates undocumented credentials, manual repairs, and specialist knowledge that make transfer impossible.

12.9 Ask AI to maintain the build record

Use an approved assistant throughout the month. Give it the charter, canvas, source manifest, decision specification, evaluation results, change log, and incident records. Ask it to maintain:

- the current version of the operating loop;
- open assumptions and their evidence status;
- differences between written authority and technical permissions;
- failures grouped by ATOM layer;
- decisions made and alternatives rejected;
- regression cases generated from each meaningful correction;
- and the evidence required at the next gate.

At each review, ask it to produce a short adversarial brief: what would have to be true for the team to be overestimating value or underestimating risk? The assistant organizes the record. The named owners accept, reject, or change the argument.

A minimum viable operating loop is ready when its authority and evidence are designed, when people can recover from failure, and when a normal owner can imagine taking it into the run. The demo is the easy part.

13. Prove Value Before You Expand

The pilot resolved cases faster. Should the company expand it?

Not yet. Faster can mean customers received good outcomes sooner. It can also mean cases were closed early and returned later, reviewers absorbed hidden work, or the easiest cases moved to the agent while people inherited a harder queue.

An operating-model pilot is not successful because the technology worked. It is successful when the new combination of people, software, agent, controls, and management produces a better result at an acceptable total cost and risk.

13.1 Compare complete systems

The relevant comparison is the previous operating path against the new one. Include:

- customer or business outcome;
- elapsed and working time across the full path;
- human review, escalation, and repair;
- model, tool, integration, and monitoring cost;
- failure severity and recovery;
- maintenance of instructions, policies, context, and evaluations;
- capacity created or consumed elsewhere;
- and effects on the people doing the remaining work.

This prevents a familiar accounting trick. The agent's handling cost falls, but the operations team spends more time correcting account state. The visible queue shrinks, but repeat contact rises. A business unit appears more profitable because shared platform and founder time remain free.

Use a unit of value tied to the loop: cost per correctly resolved case, margin per delivered order, time to qualified decision, avoidable loss prevented, or another outcome that survives contact with finance and operations. Report model cost, but do not confuse it with total cost.

13.2 Establish a believable comparison

A before-and-after chart is easy to make and easy to misread. Demand, case mix, staffing, seasonality, pricing, and customer behaviour may have changed during the pilot.

Where possible, release in phases or compare eligible cases assigned to old and new paths. If random assignment is operationally and ethically appropriate, it provides the strongest comparison. Where it is not, match cases on the factors that drive difficulty and show the remaining limitations.

Preserve the inclusion and exclusion rules. If the system handled only simple cases, compare it with similar simple cases—not the entire historical queue. Measure what happened to the excluded population as well. A pilot can improve one segment while degrading the rest through capacity shifts.

The aim is not to turn every operating review into an academic study. It is to avoid claiming causality the evidence cannot support. State whether the result is a controlled comparison, a phased operational comparison, an observed association, or an informed judgment.

13.3 Read the distribution

Average improvement can conceal concentrated harm. Segment results by case type, customer group, channel, risk, language, business unit, and degree of agent involvement. Inspect the tail.

A system that improves routine cases but mishandles rare consequential cases may still be useful if those cases can be detected and excluded. If the failure has no reliable signature, the average may not justify the exposure.

Latency deserves the same treatment. A lower median can coexist with a much worse tail. The agent resolves ordinary cases immediately but leaves ambiguous cases waiting in an under-staffed approval queue. The customer does not experience the median.

Likewise, aggregate labour savings can hide a worse job. When agents absorb simple cases, operators may face uninterrupted exceptions. Measure queue difficulty, interruption, emotional load, discretion, and repair burden. The objective is not to preserve every old task. It is to avoid economics that depend on invisible human absorption and eventual turnover.

13.4 Measure decision quality and system quality

The outcome may take time to appear. A pricing decision is immediate; margin and retention arrive later. A customer resolution closes today; recurrence appears next month. Use leading indicators, but keep them connected to the final result.

Decision quality asks whether the commitment was appropriate given the evidence and policy available at the time. System quality asks whether the loop assembled good context, used authority correctly, executed, and learned. Outcome quality asks what actually happened.

These can diverge. A sound decision can have a bad outcome because uncertainty is real. A poor decision can get lucky. Judging only the outcome encourages hindsight. Judging only compliance rewards a system that follows an unhelpful rule perfectly.

Review a sample in which decision and outcome disagree. Those cases teach the company whether to change the prediction, the judgment, or the policy.

13.5 Calculate the operating economics

Build a small model with ranges rather than fake precision. For each case or unit of work, include:

Variable costs: model and tool usage, external data, compute, per-transaction fees, and expected human handling.

Fixed and step costs: integration, evaluation, monitoring, incident response, governance, vendor minimums, and specialist support. Show how these change when volume or business units expand.

Failure costs: repair, compensation, delayed revenue, operational disruption, customer loss, security exposure, and the expected concentration of rare events.

Capacity effects: work removed, work added, bottlenecks displaced, and throughput created at the constrained part of the system.

Avoid converting every saved minute into cash unless capacity actually leaves the cost base or moves to valuable work. Ten minutes “saved” across a team may create convenience without creating an economic return. That can still be worthwhile, but it should be named accurately.

For a new business line, include the cost of shared assets. If margin depends on free platform capacity, founder intervention, or underpriced operations support, the pilot has not demonstrated a scalable business.

13.6 Use a four-way gate

The review should produce one of four decisions:

Decision	Evidence
Expand	Outcomes hold, total economics improve, controls work, and new volume is supportable
Revise	Value is plausible but a named context, design, or reliability gap is repairable
Contain	The loop works for a stable narrow segment and should remain there
Stop	Outcome, economics, control, or affected-party impact does not justify continuation

Containment is underused. A system may be excellent for a defined class of requests and unreliable elsewhere. Keeping it narrow is a sound operating decision, not a failure of ambition.

Expansion should name exactly what changes: volume, customer segment, geography, action right, financial limit, or operating hours. Each change has an owner, a new risk hypothesis, and evidence that could reverse it.

Stopping proves that the company can reject activity that does not create value. Preserve the cases, cost model, and lessons. The next loop should not have to rediscover them.

13.7 Transfer to the run

An experiment has not scaled when the founder or pilot team can operate it. It has scaled when a normal owner can run, monitor, and change it through defined interfaces.

Before transfer, name:

- the business owner and service expectation;
- the people who receive exceptions;
- the agent, context, platform, and control owners;
- the incident and recourse path;
- evaluation and operating-review cadence;
- cost and exposure budgets;
- change and release authority;
- vendor and dependency risks;
- and the conditions for containment or retirement.

Remove temporary credentials, manual joins, and founder workarounds. Train the people who receive escalations with real cases. Put maintenance into capacity planning. Record which assumptions remain unproven and when they will be revisited.

The receiving owner should have the right to refuse transfer. If the loop depends on specialist heroics or undocumented repairs, the pilot team has built a service, not an operating capability.

13.8 Ask AI to prepare the value case

Give an approved assistant the baseline, eligibility rules, case-level outcomes, event traces, cost data, human time, review reasons, and incidents. Ask it to:

1. construct comparable cohorts and show where matching remains weak;
2. calculate outcome, flow, control, and cost measures by segment;
3. expose changes in the human exception queue;
4. identify missing downstream outcomes and likely survivor bias;
5. build low, base, and high operating-cost scenarios;
6. list every assumption required to claim value;
7. generate the evidence for expand, revise, contain, and stop;
8. draft a transfer checklist and name unresolved dependencies.

Ask finance, operations, frontline staff, and the outcome owner to challenge the result. The model can join, segment, and stress-test the evidence. It cannot decide which customer harm is acceptable or whether a strategic option deserves continued funding.

The ninety-day question is not, “Did people use it?” It is, “Did this loop improve a result at acceptable total cost and risk, and can the company operate it without exceptional support?” Only a convincing answer earns more authority or scope.



Part V: Scale Without Losing the Plot

14	Fractal Units, Not Fractal Bureaucracy	63
14.1	A unit earns autonomy by closing a loop	63
14.2	The unit contract	64
14.3	Invariants and local choices	64
14.4	Design shared services as products	65
14.5	Illustrative example: two locally correct decisions	65
14.6	Decide where work belongs	65
14.7	The fractal boundary	65
14.8	Use AI to test the unit design	66
15	Allocate Capital, Talent, and Knowledge Dynamically	67
15.1	Match cadence to the decision	67
15.2	Fund options, not forecasts	67
15.3	Make opportunity cost visible	68
15.4	Protect the economics of shared assets	68
15.5	Move expertise without dissolving teams	68
15.6	Allocate management attention	68
15.7	Treat knowledge as maintained infrastructure	69
15.8	Run a monthly allocation review	69

14. Fractal Units, Not Fractal Bureaucracy

As a company adds business lines, it faces a familiar choice. Centralize decisions and slow the edges, or decentralize and accept duplication. The choice is false when the organization has not specified what must be shared.

This problem appears early. A forty-person company launches a second offer and appoints a leader. At first the new business borrows sales, product, finance, and operations from the core. Every shared person receives two sets of priorities. The founder resolves conflicts because the interfaces do not exist.

At seventy people, the company adds heads of function. The unit leader still cannot commit shared capacity or change a common product. At one hundred and fifty, leaders may respond by duplicating every function inside every business—or by adding a central planning layer to coordinate the shared ones. Both moves can be expensive ways of avoiding a design decision.

ATOM uses the term **Fractal Unit** for a semi-autonomous group that can run a complete governed outcome loop. “Fractal” does not mean every unit is identical. It means the same operating grammar recurs at different scales: intent, context, decisions, execution, evidence, learning, and control.

The repeatable object is not the team diagram. It is the contract.

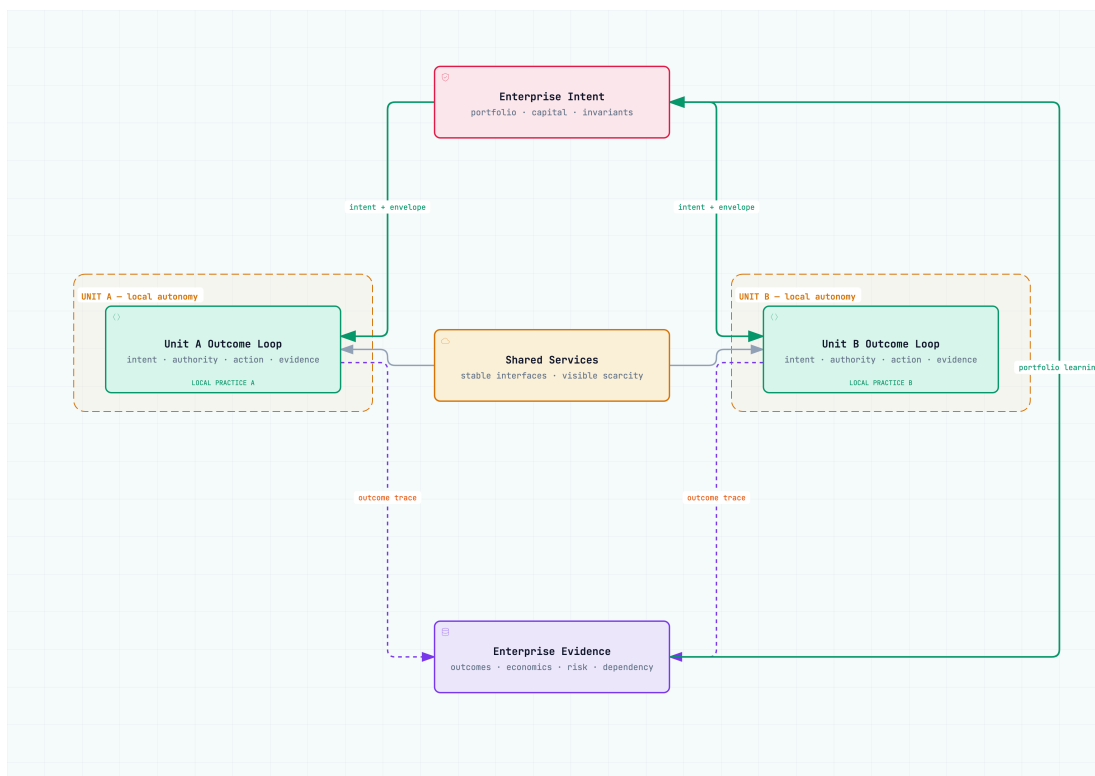


Figure 14.1: Fractal Units share enterprise intent, evidence, and stable service interfaces while retaining different local operating practices.

14.1 A unit earns autonomy by closing a loop

A collection of people does not become a unit because it receives a name, leader, and targets. It becomes an operating unit when it can observe an outcome, make relevant decisions, execute inside defined boundaries, and learn from the result.

A growth squad that depends on five functions to approve each experiment may be cross-functional on paper but not autonomous. A business line with its own revenue target but no visibility into fulfilment

cost does not own its economics. A regional team that can sell but not resolve customer failures owns acquisition, not the customer outcome.

The unit boundary should follow a result that can be owned coherently. It may be a business, product, customer segment, operational service, or platform. The choice depends on where context is deep, decisions interact, and feedback needs to remain close.

Too small a boundary produces fragments whose local goals compete. Too large a boundary recreates the central company inside each unit. The test is whether the group can run the complete loop without continuously negotiating ordinary work across its edge.

14.2 The unit contract

Every unit should be able to state:

Element	Contract
Purpose	Customer or enterprise outcome the unit exists to produce
Economics	Revenue, cost, margin, assets, or risk it owns and can observe
Inputs and outputs	What enters, what leaves, and in what form
Decisions	Rights held locally, shared, and at the centre
Dependencies	Services and capacities required from others
Context	Sources and policies required to operate
Controls	Non-negotiable limits and escalation conditions
Evidence	Measures, traces, and review obligations
Learning	What the unit may change and what requires shared approval
Lifecycle	Conditions to grow, split, merge, or close

A unit without outcome ownership is a project team. A unit without visible economics is a cost centre pretending to be a business. A unit without bounded rights is a reporting layer.

The contract is reciprocal. The unit commits to outcomes, evidence, and enterprise constraints. The centre and shared services commit to capacity, interfaces, decision latency, and transparent economics. Autonomy cannot mean that the unit is accountable for a result while a central function can silently delay the input it needs.

14.3 Invariants and local choices

Scale depends on making a few things rigid enough that everything else can adapt.

Enterprise invariants may include customer identity, financial definitions, security controls, core product interfaces, evidence requirements, and rules for allocating shared assets. Units should not invent their own meaning of active customer or contribution margin. They should not bypass access policy because the central service is slow.

Local choices include workflow, roles, experiments, tools, and decisions inside the unit's authority envelope. The centre should resist standardizing them simply because one unit found a successful pattern. Reuse should follow evidence and compatible conditions, not executive preference.

The difficult category lies between them: shared choices with local consequences. Examples include platform priorities, brand promises, field capacity, scarce expertise, and common data products. These need an owner, allocation rule, service interface, evidence, and appeal. Otherwise the shared function becomes an informal centre of power.

An invariant should be few, explicit, and enforceable. If everything is an invariant, the unit is not autonomous. If nothing is, the company becomes a portfolio of incompatible operations.

14.4 Design shared services as products

Calling a function “shared” does not define how it serves the businesses. A shared service needs a purpose, users, service levels, capacity model, cost, prioritization rule, roadmap, and owner.

Finance may provide close, cash, cost definitions, pricing context, and decision support. Those are different services with different time expectations. A product platform may provide identity, billing, data, and deployment capabilities. A physical network may provide reserved capacity, emergency capacity, and maintenance windows.

The interface should expose scarcity. If all requests appear free, demand becomes political. Units over-request, central teams triage through relationships, and the COO becomes the appeal court.

Internal pricing can help but is not the only mechanism. Capacity envelopes, priority classes, explicit queues, and opportunity-cost views may be sufficient. The objective is to make local choices feel the consequence of shared constraints without constructing a complex internal market that costs more than it clarifies.

14.5 Illustrative example: two locally correct decisions

A company adds an energy-services unit beside its original equipment business. Both use the same field engineers and device platform.

The equipment unit schedules engineers to protect installation revenue. The services unit promises response times to win recurring contracts. Each choice is rational inside its own plan. Together they oversubscribe the same people, and both units miss commitments.

The first response is a central capacity committee. Every exception moves upward. Each leader brings forecasts, customer stories, and urgency. The meeting becomes the operating system.

A stronger design exposes field capacity as a shared service with a named owner, reservation rules, priority classes, cost, and exception path. The centre sets the portfolio allocation and reliability buffer. Each unit commits inside its envelope. Evidence shows demand, utilization, missed promises, and the economic value of each use.

When an unexpected outage consumes the buffer, the shared-service owner applies the emergency rule. If the rule cannot settle the conflict, the exception goes to the portfolio owner with the affected commitments and options—not to a general coordination meeting.

The units remain different. The interface becomes stable.

14.6 Decide where work belongs

Place a capability inside a unit when it requires deep local context, changes frequently with the offer, and can be owned end to end. Share it when scale, scarce expertise, risk consistency, or a common asset creates real advantage. Keep it at the centre when the decision sets enterprise intent, allocates capital among units, defines non-negotiable policy, or resolves a conflict the unit cannot legitimately settle.

Five questions improve the choice:

1. Where does the information needed for good judgment live?
2. How often must the capability adapt to the unit’s customer or economics?
3. Does duplication create meaningful cost or merely discomfort?
4. Does divergence create enterprise risk or healthy variation?
5. Can the dependency be expressed through a stable interface?

Do not decide from ideology. Centralization has latency, queue, and context-loss costs. Decentralization has duplication, inconsistency, and fragmented-learning costs. Make both visible.

A function may also move over time. A new unit can borrow central expertise while its needs remain uncertain, then embed capability when the volume and local context justify it. Another service may centralize after units converge on a common pattern. The lifecycle belongs in the contract.

14.7 The fractal boundary

The fractal idea can become bad management if it is interpreted as endless fluidity.

Constantly reforming teams destroys relationships, tacit knowledge, and accountability. Physical assets do not reallocate like cloud capacity. Regulated responsibility may remain centralized even when execu-

tion moves. People need a home, a manager who understands their development, and enough stability to become competent together.

Nor should a small unit reproduce every corporate role. The operating grammar repeats; the bureaucracy does not. A six-person venture does not need miniature finance, legal, HR, and security departments. It needs usable interfaces to those capabilities and clear local decision rights.

The strongest boundary is often stable for the work and explicit at the interfaces. Adaptation then happens through changes in intent, authority, capacity, and practice—not through constant reorganization.

14.8 Use AI to test the unit design

Give an approved assistant the proposed unit charters, P&L definitions, shared-service catalogues, decision inventory, dependency data, escalation records, and capacity use. Ask it to:

- map decisions that supposedly belong to a unit but routinely travel to the centre;
- identify outcomes whose costs or dependencies sit outside the owner's view;
- find shared services with no service level, allocation rule, or appeal path;
- compare conflicting definitions across units;
- locate duplicate capabilities and test whether the duplication is harmful;
- model two or three boundary options and expose the tradeoffs;
- draft unit contracts with every inferred right clearly marked;
- and simulate a stress event in which two units compete for one constrained asset.

Use the result to conduct a boundary review with the unit and shared-service owners. Do not ask the model to choose the structure. It can expose inconsistency, hidden dependency, and likely failure paths. Leaders must decide which outcomes deserve autonomy and which obligations bind the enterprise together.

What repeats at scale is a governed outcome loop. When unit contracts and shared interfaces are explicit, local autonomy and enterprise coherence stop being opposites.

15. Allocate Capital, Talent, and Knowledge Dynamically

Adaptability is not the frequency with which a company changes priorities. A company that reallocates people every two weeks can be highly reactive and incapable of learning.

Dynamic allocation means that evidence can change the commitment of capital, talent, and shared capacity at a cadence matched to the reversibility of the decision. It is disciplined movement, not permanent motion.

This distinction matters in a scale-up. The core business needs reliability. New businesses need options. Shared capabilities need enough stability to improve. If every signal can rewrite every commitment immediately, the operating model becomes a contest of urgency.

15.1 Match cadence to the decision

Different resources should move at different speeds.

Cloud compute or a marketing budget can often be changed quickly. Specialist time and field capacity require notice. A product architecture, warehouse footprint, or regulated process creates longer commitments. Team relationships and domain knowledge accumulate slowly even when the organization chart can be redrawn overnight.

For each resource, define:

- the decision owner;
- the smallest useful allocation unit;
- commitment period and notice required;
- evidence that can change the allocation;
- cost of reversal;
- protected buffer;
- and the next review point.

“Continuous allocation” then means the organization can see new evidence without pretending every resource is continuously reversible.

15.2 Fund options, not forecasts

An emerging business needs enough capital to resolve uncertainty, not a five-year plan disguised as precision. Release funding in stages tied to evidence: customer behaviour, unit margin, asset use, operational feasibility, or a technical capability.

The unit has authority inside the tranche. It should not return to the executive team for every experiment that fits the agreed intent and budget. The portfolio owner decides whether the evidence justifies the next commitment.

Each tranche should answer one or two uncertainties. If the new unit does not know whether customers will pay and whether the operation can deliver profitably, one experiment may test willingness to pay while another measures delivery effort. A larger budget that blends both can produce activity without learning.

AI can compare assumptions with current signals, trace which evidence supports each claim, and simulate the effect of alternative allocations. It can expose when a unit quietly changes its success definition. It should not automatically move capital. Portfolio choices contain strategic judgment, timing, and opportunity cost that remain executive responsibilities.

Mature operations need a different cadence. Constantly reopening a factory plan, regulated process, or committed customer programme destroys value. Signals should remain visible between formal decisions, but evidence earns a review; it does not erase commitments by itself.

15.3 Make opportunity cost visible

Budgets show what a unit spends. They do not always show what the company forgoes.

A new offer consumes product attention that could improve the core. A strategic customer receives scarce field capacity. An internal agent project uses security and data specialists needed elsewhere. The direct cost may be small while the displaced option is large.

The portfolio view should show constrained resources and the choices competing for them. This does not require a single mathematical ranking. It requires comparable units, explicit assumptions, and an owner able to make the tradeoff.

Ask what would receive the resource if this option did not. If the answer is “nothing,” verify that the capacity is actually available. In growing companies, founder time and shared-team effort often appear free because no system records the work they displaced.

15.4 Protect the economics of shared assets

A multi-business company cannot delegate well if units cannot see what they consume. Shared engineering, distribution, data, brand, or physical capacity needs an explicit allocation method. It does not have to reproduce accounting perfection. It must be consistent enough that local decisions reflect real scarcity.

Useful questions include:

- Which asset constrains growth now, and which is likely to constrain it next?
- What return, learning, or obligation does each use produce?
- Which demand is committed and which remains optional?
- What buffer protects reliability?
- Which unit is exporting cost or risk to another?
- Who can override the allocation rule, and what evidence is required?

AI can detect patterns in usage, forecast pressure, and prepare allocation scenarios. The rule still needs an owner and an appeal path. A model optimizing measured return may starve a new activity whose strategic value has not yet appeared in historical data. It may also favour the mature business because its measurements are cleaner.

Keep strategic exceptions explicit and budgeted. Otherwise every unit learns to describe ordinary demand as strategic.

15.5 Move expertise without dissolving teams

Internal talent systems can make hidden capability visible and connect people to work beyond their reporting line. Used badly, they turn employees into interchangeable capacity and managers into bidders.

People are not compute. Team cohesion, domain knowledge, consent, development, and career identity are operating constraints. A specialist loan should specify purpose, duration, home team, receiving owner, protected time, learning to return, and what happens when priorities conflict.

The home team should feel the cost. If another unit receives a specialist without replacing capacity or reducing commitments, the “dynamic” allocation merely transfers an invisible queue.

The person should also see and challenge the information used to suggest the match. Project history may be incomplete. Self-declared interests may be old. Performance data may be sensitive or inappropriate for the decision. A recommendation can broaden opportunity only if it does not become an opaque assignment system.

Illustrative example. The newer business unit needs a reliability engineer for six weeks. An internal matching assistant identifies three people from project histories and declared interests. The highest-ranked candidate is carrying an incident-recovery commitment invisible to the model. The second is interested and accepts a time-bounded assignment. Her home team receives backfill capacity, and the platform lesson becomes a shared standard rather than private expertise.

The system accelerated discovery. Human context changed the decision. The contract protected both teams.

15.6 Allocate management attention

Senior attention is another constrained asset, usually managed poorly.

If the COO is copied on every exception, attends every new-business review, and resolves every cross-unit conflict, the company has not delegated the decision. It has delegated preparation. The founder's calendar becomes the control plane.

Track executive attention by decision type. Which issues return repeatedly? Which could be settled by a unit contract, context product, or shared-service rule? Which require genuine portfolio judgment? The objective is not an empty calendar. It is to spend senior attention where enterprise intent, legitimacy, or irreversible commitments are at stake.

An AI assistant can classify meeting notes, approval records, and messages to reveal recurring decision demands. The uncomfortable part is managerial: leaders must stop reopening choices they granted to others.

15.7 Treat knowledge as maintained infrastructure

Shared knowledge does not improve because the company generates more documents. It improves when reliable decisions, interfaces, policies, and lessons are reusable at the point of work.

Every knowledge product needs an owner, audience, source, freshness expectation, and retirement rule. Search logs and agent failures can reveal unanswered questions. Customer conversations can reveal where policy and reality diverge. Unit reviews can nominate a local pattern for reuse.

A shared owner decides whether the pattern becomes:

- an enterprise invariant;
- a reusable but optional template;
- a local practice that remains local;
- an evaluation case;
- or a retired approach.

This prevents “best practice” from spreading without its conditions. A sales play that worked in the core business may destroy margin in the new one. A support workflow designed for one region may violate another's obligations. Knowledge travels with evidence and scope.

The **ATOM Playbook** is the maintained set of operating knowledge. It is not a culture wiki or an archive of meetings. It contains intent statements, definitions, unit contracts, decision specifications, context products, controls, evaluation cases, and operating lessons the company expects others to reuse.

Agents can make the Playbook active. At the point of work they retrieve the relevant rule, example, or interface. After incidents they propose updates. They compare local practice with the current standard. But each change still has an owner. Generating a new page is not organizational learning.

15.8 Run a monthly allocation review

The review begins with constrained resources, not every project. Give an approved assistant the current allocations, actual consumption, unit outcomes, commitments, experiment evidence, skill requests, executive approval logs, and Playbook changes. Ask it to:

1. identify where actual use differs materially from allocation;
2. expose assets whose scarcity is hidden by queues or founder intervention;
3. compare each growth option with the next-best use of the constrained resource;
4. find tranches whose learning question was not answered;
5. map specialist loans and the commitments displaced in home teams;
6. cluster recurring executive interventions by missing decision or interface;
7. identify knowledge products that are stale, duplicated, or widely reused without an owner;
8. generate three allocation scenarios with explicit assumptions and reversibility.

The portfolio group does not accept the model's ranking. It uses the analysis to make opportunity cost visible, decides which commitments change, and records what evidence would cause the next change.

Capital moves when an option earns another commitment. Talent moves through a contract that respects teams and people. Knowledge travels after it becomes reliable enough to reuse. Adaptability comes from changing deliberately, not keeping everything in motion.

VI

Part VI: Leadership and Control

16	Agent Operations: Control at Scale .	71
17	The Manager After Coordination Gets Cheap	75
18	Accountability Cannot Be Delegated .	79
18.9	Bibliography	85

16. Agent Operations: Control at Scale

The first agent is owned by the team that built it. The fiftieth agent reveals that the company has created a new operating estate.

Agents share models, tools, identities, data, vendors, and evaluation infrastructure. They change versions. Their permissions drift. One agent's output becomes another's input. Costs accumulate across teams. A local failure can become a repeated enterprise action.

The company needs an **Agent Operations** capability. It may begin as a responsibility inside platform or security rather than a new department. Its job is to make the estate visible and operable without taking business accountability away from the units.

This is different from an AI centre of excellence. A centre often advises projects and spreads methods. Agent Operations runs common control infrastructure and the lifecycle of production actors. It does not decide which customer outcomes a unit should pursue.

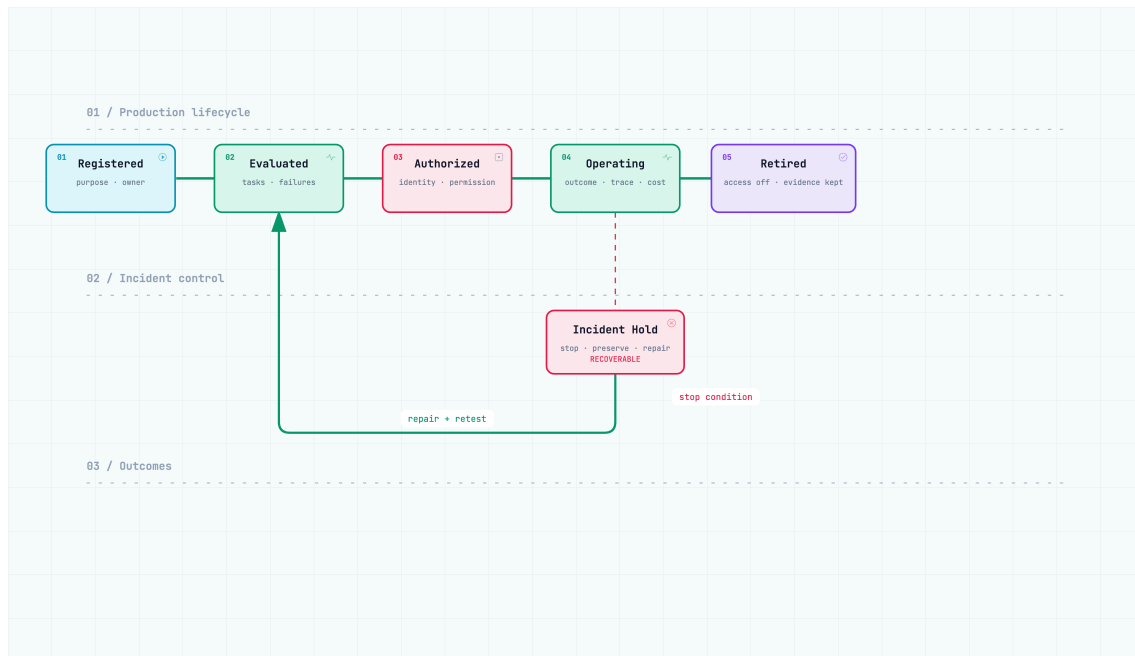


Figure 16.1: A production agent should move through registration, evaluation, authorization, operation, and retirement; incidents force a stop, repair, and retest.

16.1 Register the actors

Every production agent should have a registry entry containing:

- name, purpose, and business owner;
- current version and environment;
- models, instructions, tools, data sources, and downstream dependencies;
- identity, permissions, credential owner, and expiry;
- decisions and actions it may take;
- applicable controls and required approvals;
- evaluation suites, current results, and monitored outcomes;
- cost and exposure budgets;
- incidents, known limitations, and change history;
- stop mechanism, fallback, and retirement condition.

The entry should describe the deployed combination, not merely a model endpoint. The same model with another tool, context source, or authority is a different operating actor.

If nobody can list the agents that can modify customer, financial, code, physical, or employee state, the company does not control them. Discovery should include vendor features and departmental automations, not only systems built by the platform team. Agents can enter through a SaaS configuration, an employee's connected workspace, or a script that quietly becomes operational.

The registry must remain useful. Ownership is periodically attested. Unused credentials expire. An agent without a current owner loses production authority. A team should be able to query, "Which actors can write to the CRM and read contract data?" and receive an answer that matches reality.

16.2 Give agents identity without personhood

An agent needs a distinct technical identity so its access, actions, and costs can be attributed. That does not make it an employee or accountable person.

Avoid shared human accounts, founder credentials, and permanent broad tokens. Bind identity to a workload, environment, purpose, and owner. Separate development, testing, and production. Use short-lived credentials where possible. Record delegation: which human or service initiated the run and under which authority.

NIST's 2026 work on software-agent identity and authorization highlights identification, authorization, auditing, non-repudiation, and prompt-injection controls as open enterprise concerns [23]. The standards are still developing. The operational need is already present.

Identity enables revocation. If one agent is compromised or retired, the company can remove its access without breaking an unrelated workflow. It also makes an incident intelligible. "The integration user changed the record" is not enough when several systems share it.

16.3 Divide responsibility without dividing accountability

The **business owner** defines the outcome, authority, exclusions, and acceptable failure. **Agent Operations** provides registry, observability, evaluation infrastructure, deployment standards, and lifecycle controls. **Platform teams** operate shared models, tools, context services, and environments. **Security, risk, legal, privacy, and finance** define specialist constraints and challenge the deployment.

These roles can be held by a few people in a scale-up. What matters is that they are not collapsed into "the AI team."

A technically reliable agent can still pursue the wrong business objective. A business owner cannot waive an enterprise access control by calling the experiment urgent. Security should not become the owner of customer-resolution quality because it approved the connection. Responsibility is distributed; accountability for the outcome remains named.

NIST's Generative AI Profile organizes risk work across governance, mapping, measurement, and management throughout the lifecycle [15]. The practical lesson is to connect inventory, testing, monitoring, incident response, and ownership rather than treat them as launch paperwork.

Use a RACI chart if the organization already understands it, but add two fields it often misses: **who can technically stop the system** and **who can authorize its return**. In an incident, advisory responsibility is not enough.

16.4 Control the change surface

An agent can change even when its application code does not. The model provider updates a version. A context source changes schema or meaning. A tool adds a parameter. A policy is revised. Another agent begins consuming its output. The environment around the actor is part of its version.

Material changes should run against relevant evaluations before release. Progressive rollout limits exposure. The registry should show which units and workflows depend on the changed component. A high-consequence loop may pin versions until its owner reviews evidence; a low-consequence assistant may accept faster updates.

Define materiality before the change occurs. New write access, a different data class, higher transaction limit, new model, modified authority, changed policy, new downstream consumer, or expanded population usually qualifies. Minor wording and bug fixes may follow a lighter path if regression tests remain sound.

The change process must include rollback. “Switch back to the old model” is insufficient if the context schema and tool behaviour also changed. Preserve compatible versions and the state needed to recover partial work.

16.5 Watch autonomy in operation

Autonomy is not a static setting. It emerges from the model, product design, user behaviour, task, and permissions. The same agent may be closely supervised by a new operator and largely unattended by an experienced one.

Anthropic’s analysis of millions of interactions found that experienced Claude Code users were more likely to use auto-approval and also more likely to interrupt, while agent-initiated requests for clarification remained important [24]. The study is specific to Anthropic’s products and should not be generalized into a universal rate. It still demonstrates why approval configuration alone does not describe operational oversight.

Monitor time and actions between human interventions, but also whether the system stops appropriately, which tools it uses, the consequence accumulated during a run, and whether a human is actually able to understand and intervene. One long read-only analysis is not equivalent to five minutes of state-changing action.

At scale, use autonomy budgets: permitted action types, cumulative value, destinations, time, iterations, and scope. The budget attaches to a decision loop and owner, not to the model’s reputation.

16.6 Multi-agent systems enlarge the estate

Multiple agents can parallelize work or separate contexts and tools. They also create internal supply chains.

Each agent needs identity and scope. The orchestrator needs a limit on what it can delegate. Worker output carries provenance. A merge or synthesis step resolves conflicts. The final action remains bound to one authority, even when many agents contributed.

Do not automatically register every ephemeral worker as a separate business system. Register the production architecture, role types, permissions, and run identities at the level required for audit and revocation. The design should let an investigator reconstruct which worker accessed which source and influenced which commitment.

Coordination failure can be expensive. Workers duplicate calls, search indefinitely, consume inconsistent state, or amplify one another’s error. Put limits on spawning, recursion, cost, and time. Test incomplete, contradictory, and malicious worker outputs. A hierarchy of agents is not exempt from the interface design expected of human teams.

16.7 Run incidents as operating failures

Consider an **illustrative example**. A sourcing agent reads a malicious instruction embedded in a supplier document. It calls a procurement tool using a broad shared token and changes payment details. A second agent treats the changed record as authoritative.

The root cause is not simply “prompt injection.” The organization exposed an unnecessary tool during document analysis, failed to bind the credential to a narrow resource and action, allowed one actor to write a state another trusted, and omitted approval for a consequential change.

The immediate response revokes credentials, isolates the workflow, preserves traces, repairs affected records, and informs the relevant owners. The business response identifies exposed suppliers and payments. The technical correction separates untrusted reading from state change, narrows authority, and requires independent verification of bank-detail changes. The evaluation suite receives the incident.

Protocols such as MCP make tools easier to connect, which increases the importance of audience-bound credentials and resource-specific authorization [20]. Integration convenience is not a control model.

Incident review should classify the failure by ATOM layer. Was intent ambiguous? Context untrusted? Decision authority too broad? Execution unable to recover? Evidence incomplete? Learning assigned to the wrong owner? That classification prevents every incident from becoming a generic model problem.

16.8 Operate cost and dependency risk

Agent economics can drift. A workflow begins with a small model and three calls, then gains retries, larger context, extra graders, and a second provider. Per-case cost remains hard to see because each component reports separately.

Connect usage to the decision record and successful outcome. Track cost distributions, not only monthly totals. Set alerts for loops, unusually large contexts, repeated tool failure, and sudden changes by version or source. Include human review and vendor minimums when evaluating the operating cost.

Dependencies deserve the same visibility. Which provider outage stops the loop? Which data source lacks a fallback? Which vendor can alter a model or feature without the company's release gate? A resilient design may use a manual fallback, queued safe state, alternate model, or reduced service. It does not need redundant providers everywhere.

16.9 Retire what no longer earns its place

Agents should have sunset conditions. Usage is insufficient. A system may be popular while generating review cost, duplicating another tool, or preserving an outdated process.

At review, compare outcome, total cost, risk, and maintenance with the current alternative. Look for owners who have left, evaluations that no longer run, credentials without recent use, agents that lost their source system, and outputs no decision consumes.

Retirement means more than hiding the interface. Revoke identities, remove integrations, archive the decision and evidence required by retention policy, notify downstream consumers, and delete data or memory that no longer has a lawful and operational purpose. Update the Playbook so people do not continue relying on a retired actor.

16.10 Ask the registry hard questions

Once a month, give an approved analysis agent the registry, identity and access exports, tool catalogue, evaluation status, usage and cost, incidents, dependency graph, and business outcomes. Ask it to find:

- actors whose technical permissions exceed their registered authority;
- shared or human credentials used by agents;
- write-capable agents that consume untrusted external content;
- downstream systems trusting output without provenance;
- expired evaluations, owners, policies, or data sources;
- changes that reached production without relevant regression tests;
- concentration by model, vendor, tool, or source;
- systems whose use is high but outcome is unknown;
- and candidates for narrowing, consolidation, or retirement.

The report goes to the business and control owners. The analysis agent can expose drift; it should not grant, revoke, or expand its own authority.

Agent Operations succeeds when the company can answer quickly: Which agents can act? For whom? Using what? Under whose authority? With what evidence? At what total exposure? And how do we stop them? At scale, those are operating questions, not technical inventory trivia.

17. The Manager After Coordination Gets Cheap

Management will not disappear because an agent can summarize status. Some management work should disappear.

A manager who collects updates, reformats them, and carries information between rooms performs coordination that software can increasingly reduce. A manager who clarifies intent, designs decisions, resolves conflict, develops judgment, and repairs interfaces does work the organization needs more of as execution accelerates.

The distinction is important because companies can remove the visible administrative task without changing the system around it. The weekly report is generated automatically, but the meeting still exists because nobody trusts the measures or knows who may decide. An agent drafts performance feedback, but the manager has not observed the work closely enough to use it. Automation reveals whether the role contained judgment or merely information movement.

17.1 What cheap coordination actually changes

Generative systems can prepare briefs, compare plans with actuals, retrieve decisions, follow action items, draft communications, cluster customer feedback, and monitor routine signals. These capabilities reduce the cost of assembling and transmitting information.

They do not automatically reduce the cost of agreement, commitment, conflict, or consequence. A randomized Microsoft study across more than 6,000 workers found changes in independently alterable work patterns such as email time and document completion, but no significant change in meeting time [13]. One study cannot settle the future of management. It does show why task efficiency should not be confused with a redesigned organization.

When analysis becomes abundant, managerial attention moves toward deciding which question deserves analysis, which evidence is authoritative, what threshold justifies action, and which tradeoff belongs at which level. A faster flow of recommendations makes those choices more frequent.

The manager becomes less valuable as a relay and more valuable as an architect of local judgment.

17.2 Five accountabilities

The manager in an adaptive company has five enduring accountabilities.

Set usable intent. Translate strategy into outcomes and constraints that a team can apply without asking for preference on every case. “Grow responsibly” is not usable. A defined customer outcome, economic boundary, risk limit, and review point are.

Design decisions. Make ownership, evidence, authority, escalation, and recourse explicit. Delegation is incomplete until the receiver can make a valid choice the manager dislikes.

Maintain context. Ensure the team can reach current facts, policy, history, and customer reality. Do not become the private database that makes every decision possible.

Resolve exceptions and conflict. Handle work that cannot be settled within existing interfaces. Then decide whether the exception should remain rare or trigger a redesign.

Develop judgment. Help people understand tradeoffs rather than reproduce the manager’s answer. Use agent traces, alternatives, overrides, and outcomes as material for coaching.

These accountabilities do not require every manager to become an AI specialist. They require managers to understand the work well enough to assign authority between people and systems and to recognize when the loop is learning the wrong lesson.

17.3 Manage by exceptions without becoming reactive

Real-time signals invite constant intervention. A manager sees an alert, opens the trace, changes the priority, and asks for another analysis. The organization gains observability and loses stability.

Exception-based management needs thresholds and cadence. Some signals trigger immediate action because the consequence is acute. Others enter a daily or weekly review. Strategic evidence accumulates until a defined decision point. The manager should know which category a signal belongs to before it arrives.

Alerts also need an owner and a possible action. An alert that nobody can resolve is anxiety delivered by software. Track which alerts lead to a decision, which are repeatedly dismissed, and which reveal a broken threshold.

The manager's discipline is partly restraint. When a team acts inside its authority and evidence remains within agreed limits, visibility does not create a right to intervene.

17.4 The Synthesizer

ATOM uses one specialized coordination role. A **Synthesizer** helps an outcome loop preserve coherence across intent, context, decisions, execution, evidence, and learning. The role may be held by a product-operations lead, chief of staff, programme leader, general manager, or another operator. It is not automatically a job title.

The Synthesizer notices that a team is optimizing a proxy, an exception lacks an owner, two sources conflict, a shared dependency is unpriced, or learning never enters the Playbook. The role brings the right decision into view and improves the interface.

This role requires breadth, but not ownership of everything. The Synthesizer does not approve every exception, curate every source, or run every review. She ensures that the actual owner receives a decision-shaped problem with evidence.

Her success can be measured by what stops depending on her: fewer repeat escalations, clearer handoffs, faster resolution by legitimate owners, and more operating knowledge embedded in products and contracts. If everything depends on the Synthesizer's memory, the role has recreated the debt it was meant to remove.

17.5 Build middle management around decisions

A company moving from forty to one hundred and fifty people needs managers because coordination, development, and local judgment become real work. Adding layers before defining their rights creates messengers. Avoiding layers while every conflict returns to the founder creates bottlenecks.

Start with recurring decisions and interfaces. Give a manager an outcome, visible economics, authority, context, and a route for exceptions. Decide which choices belong to the business unit, shared function, and centre. Then choose the span that lets the manager perform that work.

Universal span-of-control ratios are weak guides. Agents can reduce reporting and preparation load and may allow wider spans in stable work. In novel, physical, regulated, or relationship-heavy operations, human attention may remain the constraint. A manager with twenty people and clean, repeatable work may have more capacity than one with six people across a new cross-functional service.

Observe the actual load:

- time spent relaying status versus improving decisions;
- number and age of escalations;
- coaching and development demand;
- interfaces owned across teams;
- operational incidents and customer consequence;
- amount of tacit context held by the manager;
- and decisions that return after delegation.

Use those signals to shape the role. Do not assume that adding an agent automatically creates room for more direct reports. The system may remove simple work while increasing the density of exceptions.

17.6 Develop judgment with traces

Traditional coaching often reviews the outcome after the reasoning has vanished. Agent-assisted work can preserve the evidence, alternatives, recommendation, decision, and later result.

A manager can use those records to ask better questions. Which fact changed your mind? Which uncertainty did you accept? Why did you override the recommendation? What would have made the decision reversible? Which part of the rule should apply next time?

This is different from monitoring every keystroke. The purpose is to improve judgment and the operating system, not to score employees by model agreement or activity. People must be able to explore, dissent, and make a defensible decision that produces an uncertain outcome.

Early data from Anthropic's Claude Code use suggests that domain expertise helps people direct agents and recover from errors, even when coding proficiency differs [25]. The finding is specific to one product and domain. It supports a broader hypothesis: as implementation becomes cheaper, understanding the problem and its consequences becomes more valuable.

Managers should therefore deepen domain judgment rather than outsource it. An agent can provide options, counterarguments, and simulations. The manager helps people recognize which model of the situation is appropriate.

17.7 Influence without hierarchy

The strategic COO works increasingly through peers who do not report to them. Authority comes from making the enterprise tradeoff legible.

Suppose sales wants capacity for a strategic account, the new business wants the same capacity for an experiment, and operations wants a reliability buffer. The COO should not arrive with more slides. She should frame the decision: the scarce asset, existing commitments, alternatives, reversibility, economic and customer consequences, and the right owner.

An AI assistant can prepare the ground. Give it each function's proposal, operating data, prior decisions, and enterprise intent. Ask it to identify incompatible assumptions, missing evidence, likely incentives, the cost of delay, and what each side would need to believe to change its position. Ask it to produce the strongest objection to the COO's preferred answer.

The meeting remains human because the tradeoff requires legitimate commitment. Better preparation reduces rhetorical conflict and makes disagreement specific.

Peer influence grows when the COO can separate enterprise constraints from personal preference. The aim is not to win every argument. It is to leave the company with a decision that others understand, can execute, and can later evaluate.

17.8 The founder's hardest delegation

The founder has given a business-unit leader a margin envelope. The leader chooses a price inside it. The founder believes another price would be better.

If the decision is sound, within authority, and supported by required evidence, taking it back teaches the organization that delegation ends at disagreement. The founder may challenge the reasoning, observe the outcome, and revise the envelope at the agreed review. They should not silently reclaim the case.

This is particularly difficult for founders whose standards built the company. Their intuition often contains real customer and product knowledge. But an unstated standard enforced through intervention cannot scale. The organization learns to anticipate taste instead of exercising judgment.

The founder must separate three reactions:

1. **The decision violates an agreed boundary.** Intervene and repair the control.
2. **The decision reveals missing enterprise context.** Add the context through its owner.
3. **The decision differs from personal preference inside the grant.** Let it stand and learn from the result.

An AI assistant can help review patterns in founder interventions. Classify the last thirty reclaimed or heavily edited decisions by violated boundary, missing context, weak judgment, or preference difference. Show which categories repeat and what system change would reduce them. The classification will not be perfect; the discomfort is still useful.

17.9 Rewrite the management week

Take one manager's calendar, approval queue, recurring reports, messages, and decision log from the last month. In an approved workspace, ask an assistant to separate the work into:

- information assembly that can be automated;
- recurring decisions that need a specification;
- approvals that add no material judgment;
- genuine exceptions requiring the manager;

- coaching and relationship work;
- cross-functional interface failures;
- and work that exists because authority is unclear.

Then redesign the week. Automate one relay. Remove one ceremonial approval. Delegate one decision with a real envelope. Reserve time for exception review, coaching with evidence, and interface repair. Set a rule for which alerts deserve immediate attention.

The manager after cheap coordination is not a supervisor of more output. The role is stewardship of the conditions under which people and agents make good commitments. That includes allowing a legitimate decision to stand when it is not the one the manager would have made.

18. Accountability Cannot Be Delegated

An agent sends the message. A manager approved the workflow. A vendor supplied the model. A platform team exposed the tool. A business unit defined the target.

When the outcome harms a customer, each party can accurately describe only part of the cause. The organization still owes an answer.

This is the final boundary of the adaptive company. Action can be distributed across people, rules, services, models, vendors, and agents. Accountability cannot disappear into the distribution.

18.1 Separate five ideas

Execution is performing an action.

Authority is legitimate permission to choose or act.

Responsibility is an assigned duty within the system.

Accountability is the obligation to explain, justify, and repair the outcome.

Liability is the legal allocation of consequences.

These ideas overlap but are not interchangeable. An agent can execute and receive bounded technical authority. It cannot hold institutional accountability. A vendor contract may allocate liability without becoming the company's moral or managerial answer to an affected person. A human reviewer may be responsible for one approval without being accountable for the decision to deploy the system.

Confusion is convenient. After a failure, the business owner points to the model, the platform team points to the approved use case, the vendor points to configuration, and the approver points to the recommendation. Each statement contains some truth. None answers why the company designed the system that way.

18.2 Accountability follows the power to change

The accountable owner must have enough authority to alter the outcome and the conditions that produce it. Naming a frontline operator accountable while policy, data, permissions, and product design sit elsewhere is unfair and ineffective.

This does not mean one executive personally performs every repair. It means one human institution owns the result and can require action from the responsible parts. For a customer-resolution loop, the business owner may be accountable while context, platform, security, and operations owners hold defined responsibilities.

Ask a simple question: after a harmful pattern is confirmed, who can stop the activity, compensate the affected party, change the decision envelope, and require the relevant system owners to repair it? If no role can do those things, accountability exists only in language.

The accountable owner also needs information. A person cannot answer for a system whose actions and versions cannot be reconstructed. Evidence is therefore not merely an audit feature. It is a precondition for meaningful accountability.

18.3 Build the accountability chain

For each consequential human-agent outcome, the company should be able to trace:

1. the board or executive intent;
2. the business owner who chose the use and target result;
3. the authority granted to each person, workflow, service, and agent;
4. the context, controls, and versions in force;
5. the actual action and evidence;
6. the person or group affected;
7. the route to challenge, correction, and remedy;
8. the owner able to change the system after the case.

The chain prevents a familiar evasion: “the model made the decision.” Models produce outputs inside systems that people select, connect, authorize, monitor, and continue to operate.

It also prevents the opposite evasion: blaming the last human who touched the case. An approver may have received a polished recommendation without the source, faced an impossible queue, or lacked authority to reject. The review should examine the design of oversight as well as the person’s action.

For uses covered by law, duties may attach to providers and deployers. The EU AI Act, for example, requires deployers of high-risk systems to assign oversight to people with the necessary competence, authority, and support, together with other technical and organizational measures [22]. The exact duty depends on role and use and requires qualified legal analysis. The operating principle is broader: oversight without competence, information, and authority is ceremonial.

18.4 The board and executive team still choose the exposure

Operational ownership does not remove enterprise responsibility. Boards and executives choose where the company permits increasingly autonomous systems to affect customers, employees, capital, infrastructure, and public obligations.

They do not need to approve every agent or understand every prompt. They do need a view of material deployments, aggregate exposure, incidents, concentration, control capability, and unresolved risks. They set risk appetite and demand evidence that the organization can stop and repair its systems.

A useful executive review asks:

- Where can agents commit material value or affect rights?
- Which outcomes lack a named business owner?
- Which controls depend on human approval at a volume humans cannot sustain?
- Where do several agents or units create aggregate exposure beyond local limits?
- Which critical loops depend on one vendor, model, source, or operator?
- Which incidents changed a decision right, control, or evaluation?
- Which systems would we be unable to explain to the affected person?

This is operating oversight, not a catalogue of experiments. The executive team should spend more time on concentration and accountability than on adoption anecdotes.

18.5 Recourse is part of the design

An affected person needs more than disclosure that AI was involved. They need a way to contest the relevant result, reach someone able to change it, and receive repair when warranted.

The route should match the consequence. A poor recommendation may need feedback and correction. A denied service, employment action, financial decision, or binding contract may require explanation, preserved evidence, independent review, and timely remedy. The person handling the appeal should not merely ask the same system again.

Recourse has an operational specification:

- which outcomes can be challenged;
- how the person reaches the channel;
- what identity and evidence are needed;
- who can pause or reverse the effect;
- what response time applies;
- how the person receives an explanation;
- how repeated challenges enter incident and learning processes.

The channel must survive automation. If a support agent handles the appeal, it must recognize that the case challenges the system itself and route to independent authority.

Recourse also improves the operating model. Challenges reveal missing context, harmful proxies, weak policy, inaccessible language, and cases absent from evaluation. The organization should learn from them without treating affected people as unpaid testers.

18.6 Illustrative failure review

The following is an **illustrative example**, not a report of a real company.

A hiring-screening system ranks a candidate below the interview threshold. A recruiter follows the recommendation. Months later, an audit finds that the system inferred seniority partly from job-title patterns that systematically undervalued experience in smaller companies.

A shallow review blames the ranking model or instructs recruiters to “apply judgment.” A real review follows the accountability chain.

The business owner chose automated ranking and the volume target. The data owner approved the inputs. The agent owner defined the evaluation population. The control owner accepted the authority boundary. The recruiting operation designed a review queue in which disagreement was difficult. Executive intent emphasized processing speed without a sufficient measure for access and recourse.

The response pauses automated rejection, preserves versions and affected records, identifies people who may have been wrongly screened, offers a new review, and gives recruiters a workable route to restore candidates. The evaluation set is rebuilt across company types and career paths. The allowed features and authority are reconsidered. The business owner reports what changed and why the system may—or may not—return.

No single person caused the failure. Accountability still has an address.

18.7 When a vendor is inside the chain

Companies will depend on external models, data, platforms, and agent services. Vendor responsibility matters, but the buyer still chooses how the component enters its operating loop.

Before a consequential deployment, know which changes the vendor can make, which evidence it provides, how incidents are communicated, how data is handled, how versions are controlled, what service is available during failure, and how the company exits.

Contractual assurances do not replace local evidence. A model provider may test general capability and safety; only the deploying company knows its customer context, decision right, connected tools, and failure consequence. A shared-responsibility statement is useful only when each responsibility has an owner and an operational test.

If the system cannot function without a vendor but the company cannot pause it, obtain traces, or restore state, dependency has become loss of control.

18.8 Capability does not dissolve obligation

Agents are already being given longer tasks and more latitude in practice. Anthropic’s 2026 autonomy study found changing patterns of auto-approval, interruption, and agent-initiated clarification in its own products [24]. The authors explicitly caution that autonomy emerges from the model, user, and product design rather than belonging to the model alone.

As capability rises, companies may move human attention from individual actions to authority design, post-deployment monitoring, exception review, and incident response. That can be a stronger form of control than approving every step, provided people can understand the system, intervene in time, and repair consequences.

One condition must remain true: a named human institution can explain why the system was permitted to act and can intervene on behalf of those affected. If nobody can do so because autonomy is distributed across vendors, agents, and policies, the organization has not transcended management. It has abandoned accountability.

18.9 Conduct the accountability drill

Choose one consequential live loop and give an approved assistant its registry entry, decision specification, authority grant, sample traces, policies, vendor terms, incident plan, and recourse materials. Ask it to answer:

1. Who can stop the system immediately?
2. Who can authorize restart?
3. Who can reverse or compensate for the outcome?
4. Which owner can change context, tools, policy, and decision rights?
5. What evidence would be missing during an investigation?
6. Where could each participant plausibly deflect responsibility?
7. Can an affected person reach independent review?
8. Which vendor dependency limits explanation, recovery, or exit?

Then run a tabletop incident. Remove one critical source, allow an excessive cumulative action, simulate a vendor change, and introduce a credible customer challenge. Require the team to identify affected cases, stop exposure, communicate, repair state, and change the correct artifact.

If the exercise ends with “we would ask the AI team,” the accountability chain is incomplete.

Companies may delegate action to software. They cannot delegate the duty to justify, repair, and answer for the consequences.

Conclusion: Build One Honest Loop

The adaptive company does not begin with an announcement.

It begins when an operating leader chooses one consequential decision that repeatedly returns, waits, or fails. The company makes the intended outcome explicit. It assembles authorized context. It states who may decide and within what boundary. It assigns work to people, deterministic software, workflows, and agents. It requires evidence of the path and the result. It gives learning to owners who can change the right part of the system.

Then it watches what happens.

The first loop will reveal uncomfortable facts. Delegation may be nominal. A trusted data source may be stale. A shared function may enforce policy without providing a usable service. An agent may remove work from one team and create review elsewhere. A founder’s intervention may remain the fastest short-term answer and the largest long-term constraint.

That evidence is the work.

Do not begin with an enterprise autonomy target. Do not count generated artifacts as value. Do not build a central AI organization that owns everybody else’s outcomes. Do not use observability to rank employees or governance to disguise fear of delegation.

Build one loop whose result matters. Bound the consequence. Keep the evidence. Let a real owner make a real decision. When the result is poor, repair the context, right, interface, control, or assumption that produced it. When it is good, prove that normal operations can sustain it before expanding.

ATOM is a model, not the destination. Its value lies in the questions it forces the company to answer:

- What outcome are we trying to create?
- What context is authorized and current?
- Who or what may commit us?
- What can act, and what must stop?
- What evidence will prove the result?
- Who may change the system?
- Who remains accountable?

Answer those questions for one recurring decision and the company has begun to change its operating system.

Build one honest loop. Learn from it. Then decide what deserves to repeat.

Field Kit

The following compact templates are designed for use with an approved AI assistant and the company's existing tools. Adapt the depth to the consequence of the work.

1. ATOM loop canvas

Layer	Complete this statement
Intent	We seek ____ by ____ while protecting ____
Context	The required sources are ; owners/freshness are
Decision	____ may decide ____ within ; escalate when
Execution	People do ; rules do ; agents do ____
Evidence	We retain ____ and measure the outcome through ____
Learning	____ reviews ____ and may change ____
Control plane	Identity, permissions, approval, stop, rollback, and recourse are ____

2. Operational-debt analysis prompt

Analyze the attached anonymized workflow records. Reconstruct trigger, path, waiting, handoffs, missing context, escalations, reversals, exceptions, controls, and outcome. Separate observed evidence from inference. Link every pattern to source cases. Propose a metric, owner, repayment experiment, and alert for each material liability. Do not rank employees or invent unavailable data.

3. Decision specification

Field	Definition
Decision	The commitment being made, not the meeting
Outcome protected	Customer, economic, quality, risk, or strategic result
Accountable owner	Named human role
Inputs	Required facts and authorized sources
Criteria	Tradeoffs, thresholds, and prohibited conditions
Consequence	Severity, scale, and affected parties
Reversibility	How and at what cost the action can be undone
Authority	Human, rule, workflow, and agent rights
Evidence	Rationale, sources, action, approval, and outcome
Escalation/recourse	Trigger, destination, response expectation
Review	When the decision design expires or changes

4. Agent autonomy matrix

Level	Grant	Required evidence
Assist	Retrieve, summarize, draft	Sources and output

Level	Grant	Required evidence
Propose	Recommend an action	Options, rationale, uncertainty
Prepare	Create the exact action for approval	Full parameters and checks
Act and report	Execute reversible action inside policy	Trace, result, notification
Act within budget	Manage bounded multi-step work	Continuous controls, outcome, exceptions

Increase authority by action and condition, never by giving the agent a general promotion.

5. Context envelope

Record task state, enterprise facts, policy, relevant history, provenance, and access. For each source add owner, scope, freshness, precedence, sensitive fields, and behavior when missing or conflicting.

6. Pilot selection card

Compare value, frequency, latency, evaluability, context readiness, reversibility, consequence, integration load, ownership, and transfer to the run. Reject a candidate that cannot produce observable learning even if the demonstration is impressive.

7. First 30 days

- Days 1-5: charter, historical trace, baseline, outcome, kill condition.
- Days 6-10: decision design, context envelope, handoff contracts.
- Days 11-15: test environment, permissions, logs, evaluations, failure rehearsal.
- Days 16-20: shadow operation and calibrated review.
- Days 21-25: limited release with pause and escalation.
- Days 26-30: expand, revise, contain, or stop.

8. Evidence scorecard

Review outcome, path time, handoffs, human intervention, reversals, failure severity, recovery, model/tool cost, maintenance, affected-party feedback, and evidence completeness. Segment results and inspect tail cases.

9. Fractal Unit contract

State purpose, economics, inputs, outputs, decisions, dependencies, context, controls, evidence, learning rights, and lifecycle. Mark each element as local choice, shared interface, or enterprise invariant.

10. Agent registry

Record owner, purpose, version, environment, models, tools, sources, dependencies, identity, permissions, actions, controls, evaluations, outcomes, cost, incidents, stop mechanism, and retirement condition.

11. Ninety-day gate

Choose one: expand, revise, contain, or stop. Record the outcome evidence, complete cost, risk, distribution of effects, operating owner, unresolved assumptions, and exact authority or scope change.

12. Quality gate

Before expanding a loop, score originality of the design, clarity, evidence, practical usefulness, specificity, flow, internal consistency, and absence of inflated AI language. Any weak dimension requires a named revision, not a lower standard.

Bibliography

- [1] Stanford Institute for Human-Centered Artificial Intelligence, “Artificial Intelligence Index Report 2025,” 2025. Accessed: Sep. 11, 2026. [Online]. Available: https://hai.stanford.edu/assets/files/hai_ai-index-report-2025_chapter1_final.pdf
- [2] B. Cottier, B. Snodin, D. Owen, and T. Adamczewski, “LLM Inference Price Trends.” Accessed: Sep. 11, 2026. [Online]. Available: <https://epoch.ai/data-insights/llm-inference-price-trends>
- [3] J. Gans, *The Microeconomics of Artificial Intelligence*. The MIT Press, 2025. doi: 10.7551/mitpress/15248.001.0001.
- [4] Model Evaluation & Threat Research, “Task-Completion Time Horizons of Frontier AI Models,” 2026. Accessed: Sep. 11, 2026. [Online]. Available: <https://metr.org/time-horizons/>
- [5] Stanford Institute for Human-Centered Artificial Intelligence, “The 2026 AI Index Report,” Apr. 2026. Accessed: Sep. 11, 2026. [Online]. Available: <https://hai.stanford.edu/ai-index/2026-ai-index-report>
- [6] E. Brynjolfsson, D. Li, and L. R. Raymond, “Generative AI at Work,” *The Quarterly Journal of Economics*, vol. 140, no. 2, 2025, doi: 10.1093/qje/qjae044.
- [7] J. Becker, N. Rush, B. Barnes, and D. Rein, “Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity,” Jul. 2025. Accessed: Sep. 11, 2026. [Online]. Available: https://metr.org/Early_2025_AI_Experienced_OS_Devs_Study-paper.pdf
- [8] Model Evaluation & Threat Research, “Update: How Much Does AI Speed Up Experienced Developers?” Accessed: Sep. 11, 2026. [Online]. Available: <https://metr.org/blog/2026-02-24-uplift-update/>
- [9] DORA, “State of AI-Assisted Software Development 2025,” 2025. Accessed: Sep. 11, 2026. [Online]. Available: <https://dora.dev/research/2025/dora-report/>
- [10] Anthropic, “Demystifying Evals for AI Agents.” Accessed: Sep. 12, 2026. [Online]. Available: <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
- [11] Intercom, “Fin AI Agent Reporting.” Accessed: Sep. 12, 2026. [Online]. Available: <https://www.intercom.com/help/en/articles/7837533-fin-ai-agent-reporting>
- [12] LangChain, “Discover Errors and Usage Patterns with Insights.” Accessed: Sep. 12, 2026. [Online]. Available: <https://docs.langchain.com/langsmith/insights>
- [13] E. Dillon, S. Jaffe, N. Immorlica, and C. Stanton, “Shifting Work Patterns with Generative AI,” Apr. 2025, Accessed: Sep. 12, 2026. [Online]. Available: <https://www.microsoft.com/en-us/research/publication/shifting-work-patterns-with-generative-ai/>
- [14] Anthropic, “Effective Context Engineering for AI Agents.” Accessed: Sep. 12, 2026. [Online]. Available: <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- [15] C. Autio *et al.*, “Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile,” NIST AI 600–1, Jul. 2024. doi: 10.6028/NIST.AI.600-1.
- [16] OpenAI, “Designing AI Agents to Resist Prompt Injection.” Accessed: Sep. 12, 2026. [Online]. Available: <https://openai.com/index/designing-agents-to-resist-prompt-injection/>
- [17] OpenAI, “A Practical Guide to Building Agents.” Accessed: Sep. 12, 2026. [Online]. Available: <https://openai.com/business/guides-and-resources/a-practical-guide-to-building-ai-agents/>
- [18] Anthropic, “Building Effective Agents.” Accessed: Sep. 12, 2026. [Online]. Available: <https://www.anthropic.com/engineering/building-effective-agents>
- [19] Anthropic, “How We Built Our Multi-Agent Research System.” Accessed: Sep. 12, 2026. [Online]. Available: <https://www.anthropic.com/engineering/multi-agent-research-system>
- [20] Model Context Protocol, “Authorization.” Accessed: Sep. 12, 2026. [Online]. Available: <https://modelcontextprotocol.io/specification/2025-06-18/basic/authorization>
- [21] European Commission, “AI Act.” Accessed: Sep. 12, 2026. [Online]. Available: <https://digital-strategy.ec.europa.eu/en/policies/regulatory-framework-ai>
- [22] European Parliament and Council of the European Union, “Regulation (EU) 2024/1689 Laying Down Harmonised Rules on Artificial Intelligence, Consolidated Version.” Accessed: Sep. 12, 2026. [Online]. Available: <https://eur-lex.europa.eu/eli/reg/2024/1689/2026-07-27/eng>
- [23] National Institute of Standards and Technology, “New Concept Paper on Identity and Authority of Software Agents.” Accessed: Sep. 12, 2026. [Online]. Available: <https://www.nist.gov/news-events/news/2026/02/new-concept-paper-identity-and-authority-software-agents>
- [24] M. McCain, T. Millar, S. Huang, and others, “Measuring AI Agent Autonomy in Practice.” Accessed: Sep. 12, 2026. [Online]. Available: <https://www.anthropic.com/research/measuring-agent-autonomy>
- [25] Anthropic, “How Claude Code Is Used in Practice.” Accessed: Sep. 12, 2026. [Online]. Available: <https://www.anthropic.com/research/claude-code-expertise>